N054 Langfuse로 에이전트 문제 찾기_학습서

Chapter01

Langfuse로 에이전트 문제 찾기

지금까지 만든 AI 친구 에이전트 서비스를 동료 다섯 명에게 일주일 동안 써 보라고 했더니, 단체 대화방에 이런 말들이 모였다고 해 봅시다.

한 동료는 "어제 저녁에 도쿄 날씨를 물었는데 엉뚱한 답이 나왔어요"라고 했고, 다른 동료는 "답이 가끔 한참 있다가 와요"라고 했습니다. "다음 주에 이사 간다고 말했는데 기억을 못해요"라는 말도 있었고, "어제 업데이트한 뒤로 답이 느려진 것 같아요"라는 말도 있었습니다. 별로인 답이 몇 번 있었다는 동료도 있었지만, 언제였는지는 기억하지 못했습니다. 그사이 LLM 제공사의 청구 화면을 열어 보니, 이번 주 요금이 보낸 메시지 수에 비해 생각보다 많이 나와 있었습니다.

지금 손에 있는 것은 Cloud Run의 서버 로그뿐입니다. 로그에는 `POST /api/chat 200` 이 수백 줄 찍혀 있고, 대화 원문은 개인정보라서 로그에 남기지 않기로 했습니다. 로그로는 어느 요청이 그 동료의 대화였는지, 모델이 어떤 도구를 골랐고 무엇을 보고 답을 만들었는지, 어느 단계가 시간과 돈을 썼는지 알 수 없습니다. 오늘은 메시지 하나가 거치는 단계를 트레이스로 남기는 LLM 관측 도구 Langfuse를 붙이고, 동료들이 말한 상황을 하나씩 다시 만들어 Langfuse의 어떤 기능으로 원인을 확인할 수 있는지 찾아갑니다.

주관식 퀴즈

Q. [시작 질문] 위에서 동료들이 한 말이나 늘어난 요금 가운데 하나를 골라, 원인을 찾으려면 어떤 정보가 필요할지 적어 보세요. 그리고 지금 서비스에서 그 정보를 어디서 얻을 수 있는지(또는 없는지) 함께 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. LLM 서비스가 일반 프로그램과 다른 점을 들어 LLMOps가 필요한 이유를 설명하고, Langfuse의 트레이스와 관찰 단위(span, generation)가 무엇이며 기록이 어떻게 모여 보내지는지 말할 수 있습니다.

2. 운영 중에 들어온 문제를 보고 Langfuse의 어느 화면과 기능으로 확인할지, 그러려면 무엇을 미리 기록해 둬야 하는지 고를 수 있습니다.
3. 한 턴의 단계를 어떤 관찰 단위로 남기고 무엇을 가릴지 계측 구조를 먼저 정하고, 코딩 에이전트에게 그 구조대로 계측을 맡길 수 있습니다.
4. 트레이스를 거슬러 올라가며 읽어 엉뚱한 답, 느린 답, 늘어난 비용이 처음 생긴 단계를 찾을 수 있습니다.
5. 사용자, 세션, 환경, 버전을 붙여 한 사람의 대화와 배포 버전별 트레이스를 나눠 볼 수 있습니다.
6. 화면의 좋아요, 별로예요, 신고를 Langfuse 점수로 모아 먼저 읽을 트레이스를 고를 수 있습니다.

Chapter02

로그로는 보이지 않는 것

서버 로그는 서버가 일어난 일을 한 줄씩 남긴 기록입니다. 요청이 들어왔는지, 몇 초 걸렸는지, 오류가 났는지는 로그로 충분히 알 수 있습니다. 그런데 섹션 1에서 동료들이 한 말에는 오류가 났다는 말이 하나도 없습니다. LLM을 쓰는 서비스의 문제는 대부분 "요청은 성공했는데 답이 이상하다"는 모양으로 나타나고, 응답 코드는 200이니 로그에는 아무 흔적이 남지 않습니다.

에이전트 서비스의 메시지 하나는 안전장치, 도구 선택, 도구 실행, 답 만들기, 기억 정리의 다섯 단계를 거치고, 그 가운데 세 단계에서 모델을 부릅니다. 답이 이상했다면 모델이 도구를 잘못 골랐는지, 도구가 실패했는지, 모델이 도구 결과를 무시하고 지어냈는지, 기억이 엉뚱하게 들어갔는지를 나눠 봐야 합니다. 이렇게 요청 하나 안의 단계를 부모와 자식으로 묶고, 단계마다 입력, 출력, 걸린 시간을 붙여 남긴 기록을 트레이스(trace)라고 합니다. 트레이스를 남기도록 코드에 기록 지점을 넣는 일은 계측(instrumentation)이라고 합니다.



그림: 같은 메시지를 서버 로그와 LLM 트레이스로 남겼을 때의 차이

그림의 왼쪽 로그에서는 날씨 조회가 실패했다는 줄과 대화 요청이 성공했다는 줄이 다른 사용자의 요청과 섞여 있어, 두 줄이 같은 메시지인지조차 확신할 수 없습니다. 오른쪽 트레이스에서는 도구 실행이 실패한 것과 그 뒤에 답 만들기가 이어진 것이 한 묶음으로 보이고, 답 만들기의 출력을 열면 모델이 날씨를 지어냈는지 바로 확인할 수 있습니다. 대화 원문을 서버 로그에 남기지 않는 규칙은 그대로 두고, 원문이 필요한 기록은 접근 권한이 따로 걸린 관측 도구에만 남긴다는 점도 차이입니다.

LLMOps가 필요한 이유

동료들이 한 말을 다시 보면, 일반 프로그램에서는 잘 생기지 않는 LLM 서비스만의 문제가 섞여 있습니다. 답이 실행마다 달라지고, 이상한 답에 오류 메시지가 없고, 요금이 메시지 수로 정해지지 않고, 좋은 답인지 통과와 실패로 가를 수 없다는 점입니다. 하나씩 봅니다.

실행할 때마다 달라지는 답

일반 프로그램은 결정론적(deterministic)입니다. 같은 입력을 넣으면 언제나 같은 출력이 나옵니다.
LLM은 비결정론적(non-deterministic)입니다. 모델은 다음에 올 단어를 확률에 따라 고르므로, 같은 "오늘 좀 지쳤어"에도 어떤 때는 위로로, 어떤 때는 되묻는 말로 답합니다. 그래서 한 번 보내 보고 잘 됐다는 것만으로는 서비스가 늘 그렇게 답한다고 말할 수 없습니다. 도쿄 날씨 질문도 지금 다시 보내면 제대로 답할 수 있으므로, 동료가 겪은 바로 그 실행의 기록이 남아 있지 않으면 원인을 찾을 수 없습니다.

오류 없이 나오는 이상한 답

일반 프로그램은 오류가 나면 스택 트레이스(오류가 난 파일과 줄 번호를 보여 주는 기록)와 로그로 원인을 찾습니다. LLM 서비스의 이상한 답에는 오류 메시지가 없어서, 그 순간 모델에 무엇이 들어갔고 무엇이 나왔는지가 남아 있어야 원인을 찾을 수 있습니다. 그래서 모델을 부를 때마다 다음 다섯 가지를 남겨 둡니다.

- 전체 프롬프트: 시스템 프롬프트, 사용자 메시지, 함께 넘긴 기억과 도구 결과
- 모델 설정: 모델 이름, `temperature`(답을 얼마나 다양하게 할지), `max_tokens`(답의 최대 길이)
- 응답이 나오기까지 걸린 시간
- 입력과 출력의 토큰 수와 비용
- 에이전트가 거친 단계의 순서: 어떤 도구를 골랐고 그 도구가 무엇을 돌려줬는지

미리 알기 어려운 비용

LLM API는 토큰 단위로 요금을 받습니다. 토큰은 모델이 글을 잘라 세는 단위로, 글자 수와 정확히 같지 않고 모델마다 자르는 방식도 다릅니다. 입력 토큰과 출력 토큰에 값을 따로 매기고, 같은 토큰이라도 모델마다 단가가 다릅니다. 에이전트는 메시지 하나에 모델을 여러 번 부른다는 점이 더해집니다. 이 서비스는 메시지 하나에 도구 선택, 답 만들기, 기억 정리로 모델을 세 번 부르므로, 동료 다섯 명이 하루 스무 개씩 보내면 일주일에 메시지 700개, 모델 호출 2,100번이 됩니다. 답 만들기의 입력에는 최근 대화와 기억이 함께 들어가서, 대화가 길어질수록 호출 한 번의 입력 토큰도 늘어납니다. 호출마다 토큰과 비용을 남겨 두지 않으면 청구 화면을 연 뒤에야 요금이 늘어난 것을 알게 됩니다.

통과와 실패로 갈리지 않는 품질

일반 프로그램은 단위 테스트로 맞는지 틀린지를 바로 가립니다.

```
assert add(2,3)==5# 통과 아니면 실패
```

LLM의 답에는 정답이 하나로 정해져 있지 않습니다. "이사 준비 때문에 정신이 없어"에 대한 답이 좋은지 보려면 날씨처럼 도구 결과가 있을 때 사실이 맞는지, 앞선 대화와 기억을 반영했는지, 정해 둔 말투를 지켰는지, 해로운 내용이 없는지, 비슷한 메시지에 비슷한 수준으로 답하는지를 함께 따져야 합니다. 이 기준은 사람마다 판단이 조금씩 달라서, 무엇을 좋은 답으로 볼지와 점수를 어떻게 매길지를 따로 정해야 합니다.

서비스 중에 지켜볼 지표

개발할 때 잘 되던 서비스도 실제 사용자가 늘고 메시지가 다양해지면 문제가 생깁니다. 그래서 서비스를 운영하는 동안 다음 지표를 계속 봅니다.

지표	보는 것	동료들의 말에서
응답 품질	사용자의 반응, 평가 점수	언제였는지 모르는 별로인 답
지연 시간	메시지를 보내고 답을 받기까지 걸린 시간	가끔 느린 답, 업데이트 뒤 느려진 답
비용	하루, 한 달 동안 쓴 토큰과 요금	메시지 수보다 많은 요금
오류율	LLM API 호출 실패, 시간 초과	오늘은 들어오지 않았지만 늘 봐야 하는 값
사용 패턴	어떤 도구와 기능을 많이 쓰는지	도쿄처럼 지원하지 않는 지역을 얼마나 묻는지

LLMOps와 Langfuse

LLM 서비스를 운영하면서 위의 문제를 다루는 일을 묶어 LLMOps(Large Language Model Operations)라고 부릅니다. 머신러닝 모델을 직접 학습시키고 배포하는 MLOps와 달리, LLMOps는 이미 학습된 모델에 무엇을 어떻게 물을지(프롬프트와 컨텍스트)와 그 결과를 관리하는 데 무게가 있습니다. 모델을 다시 학습시키는 대신 프롬프트를 고치고 비교하고, 토큰 비용을 관리하고, 에이전트가 거친 단계를 추적하는 일이 중심입니다.

Langfuse는 이 일을 한곳에서 하는 오픈소스 LLMOps 플랫폼입니다. 특정 LLM 제공사나 프레임워크에 묶이지 않아서, LLM API를 직접 부르는 코드에도, LangChain 같은 프레임워크로 만든 코드에도 붙일 수 있습니다.

기능	하는 일	LLMOps에서 맡는 일
트레이싱	모델 호출과 코드의 단계를 트레이스로 기록하고 화면에서 펼쳐 봄	관측(observability)
비용 추적과 대시보드	토큰 수로 비용을 계산하고, 비용과 지연 시간과 점수를 그래프로 모아 봄	모니터링
점수	사람의 피드백이나 자동 채점 결과를 트레이스에 숫자나 범주로 붙임	평가(evaluation)
프롬프트 관리	프롬프트를 코드 밖에 버전으로 저장하고, 서비스가 실행 중에 가져다 씀	프롬프트 관리
데이터셋과 실험	입력과 기대 결과를 모아 두고, 프롬프트나 코드의 버전끼리 같은 입력으로 비교	실험

동료들의 말과 Langfuse 기능

섹션 1에서 동료들이 한 말을 하나씩 놓고, Langfuse의 어디를 열면 답을 찾을 수 있는지와 그 화면이 채워지려면 무엇을 미리 기록해 둬야 하는지를 짝지으면 아래와 같습니다.

들어온 말	확인할 곳	미리 기록해 둘 것	섹션
엉뚱한 날씨 답	트레이스 하나의 단계별 입력과 출력	한 턴의 단계를 관찰 단위로 계측	4, 5
가끔 느린 답	트레이스 목록의 지연 시간 정렬, 단계별 시간 막대	계측	6
메시지 수보다 많은 요금	generation의 토큰과 비용, 대시보드	계측, generation의 모델 이름과 토큰 수	6
잊어버린 이사 이야기	사용자와 세션 화면, 기억 정리 단계의 출력	사용자 ID, 세션 ID	7
업데이트 뒤 느려진 답	환경과 버전으로 거른 트레이스 목록	환경, 배포 버전	7
언제였는지 모르는 별로인 답	점수로 거른 트레이스 목록	화면의 반응을 점수로 보내기	8

셋째 열을 보면 오늘의 순서가 정해집니다. 계측만 해 두면 엉뚱한 날씨 답, 느린 답, 늘어난 요금까지는 볼 수 있지만, 잊어버린 이사 이야기와 업데이트 뒤 느려진 답은 누가 보냈는지와 어느 버전이 만들었는지를 트레이스에 더 붙여야 보이고, 별로인 답은 화면의 반응을 점수로 보내야 보입니다. 셋째 열은 문제가 생긴 뒤에는 채울 수 없다는 점도 눈여겨봅니다. 기록하지 않은 지난주의 대화는 지금 계측을 붙여도 볼 수 없으므로, 오늘은 동료들이 겪은 상황을 내 컴퓨터와 staging에서 다시 만들어 확인합니다. 표에 나온 관찰 단위와 generation은 바로 아래에서 설명합니다.

트레이스와 관찰 단위

Langfuse의 기록은 트레이스와 관찰 단위 두 가지로 이루어집니다. 트레이스는 요청 하나가 서비스를 지나가는 처리 전체입니다. 에이전트 서비스에서는 사용자가 메시지 하나를 보내 답을 받을 때까지가 트레이스 하나이고, 트레이스마다 ID, 이름, 입력과 출력, 누가 보냈는지 같은 값이 붙습니다. 관찰 단위(observation)는 트레이스 안의 작업 하나입니다. 안전장치 검사, 도구 선택, 날씨 API 부르기, 답 만들기가 각각 관찰 단위이고, 관찰 단위 안에 다른 관찰 단위가 들어갈 수 있어서 트레이스는 나무 모양이 됩니다. 관찰 단위의 기본 종류는 세 가지입니다.

종류	뜻	에이전트 서비스에서의 예
span	일반 작업. 입력, 출력, 걸린 시간을 남김	안전장치 검사, 날씨 API 부르기

종류	뜻	에이전트 서비스에서의 예
generation	모델 호출. span에 더해 모델 이름, 입력과 출력 토큰 수, 비용을 남김	도구 선택, 답 만들기, 기억 정리
event	시간이 걸리지 않는 한 시점의 기록	"기억 세 개를 불러옴"

Langfuse는 이 밖에도 agent, tool, chain, retriever, evaluator, embedding, guardrail 같은 종류를 지원합니다. 에이전트의 한 바퀴, 도구 호출, 문서 검색, 채점, 안전 검사처럼 단계의 역할을 화면에 드러내는 이름입니다. 토큰과 비용이 계산되는 것은 generation과 embedding이고, 나머지는 span과 같은 값을 담습니다. 오늘은 비용을 나눠 보는 데 필요한 generation과, 그 밖의 단계를 담는 span 두 가지로 계측합니다.

함수에 붙이는 계측과 자동 중첩

Langfuse Python SDK에서는 함수 위에 `@observe()` 한 줄을 붙이면 그 함수가 실행될 때마다 관찰 단위 하나가 기록됩니다. 함수 위에 `@` 로 붙여 함수의 동작을 덧붙이는 표시를 파이썬에서는 데코레이터(decorator)라고 합니다. 모델을 부르는 함수에는 `as_type="generation"` 을 더합니다. 아래는 구조를 보여 주려고 줄인 예이고, 실제 계측은 섹션 5에서 코딩 에이전트에게 맡깁니다.

```
from langfuseimport observe

@observe()
defcheck_safety(message):
...

@observe(as_type="generation")
defmake_reply(message, tool_result):
...# 여기서 모델을 부른다

@observe()
defrun_turn(message):
    check_safety(message)
    tool_result= run_tool(message)
return make_reply(message, tool_result)
```

가장 바깥의 `run_turn` 이 실행되면 트레이스가 하나 만들어지고, 그 안에서 부른 `check_safety` 와 `make_reply` 는 따로 지정하지 않아도 `run_turn` 의 자식으로 기록됩니다. 이것을 자동 중첩이라고 합니다. 한 턴을 모두 계측하면 트레이스는 아래 같은 모양이 됩니다. 토큰 수는 설명을 위한 예입니다.

```
트레이스: run_turn
├── span: check_safety
├── generation: choose_tools      입력 820, 출력 40 토큰
```

```
├── span: run_tool                 결과: 지원하지 않는 지역
├── generation: make_reply         입력 1,950, 출력 120 토큰
└── generation: update_memories    입력 1,100, 출력 60 토큰
```

도쿄 날씨 질문이었다면 `run_tool` 의 결과와 `make_reply` 의 입력을 나란히 열어, 모델이 "지원하지 않는 지역"이라는 결과를 받고도 날씨를 지어냈는지 볼 수 있습니다. 메시지 하나의 비용은 generation 세 개의 비용을 합한 값이고, 그중 어느 단계가 가장 많이 썼는지도 나눠 볼 수 있습니다. 함수 이름이 그대로 관찰 단위 이름이 되는 점과 어느 함수를 계측할지는 섹션 4에서 다시 정합니다.

데이터 모델

트레이스는 다시 몇 가지 단위로 묶입니다. 누가 보냈는지를 뜻하는 사용자(user)와 이어지는 대화 묶음인 세션(session)으로 트레이스를 모아 볼 수 있고, 사람이나 프로그램이 매긴 평가는 점수(score)로 트레이스에 붙습니다.

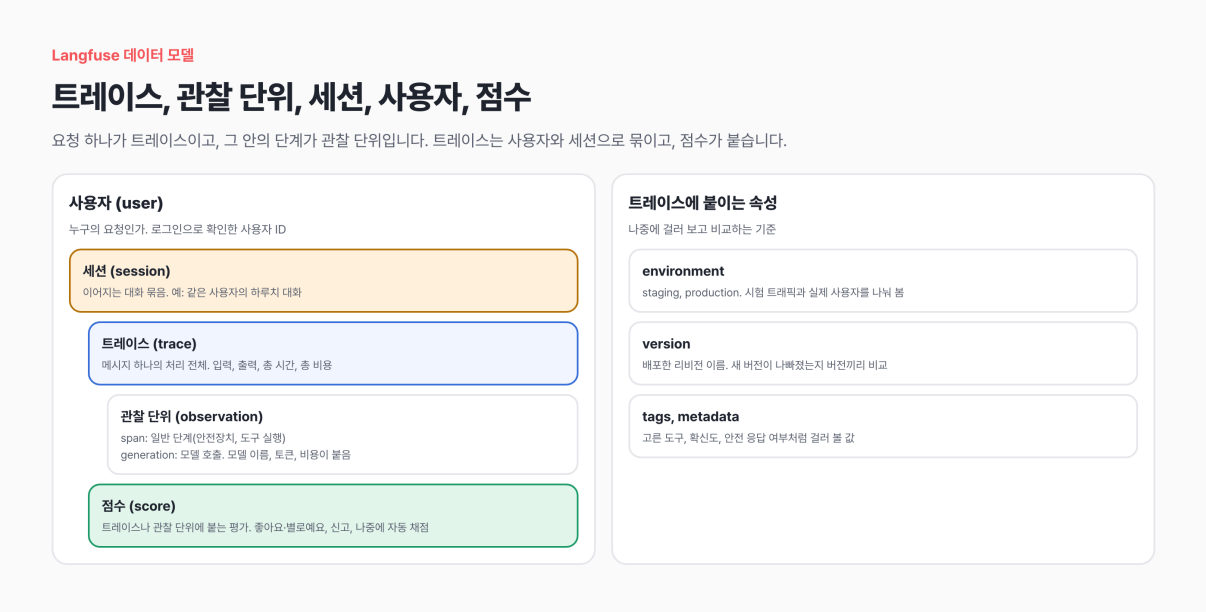


그림: Langfuse의 트레이스, 관찰 단위, 세션, 사용자, 점수와 트레이스에 붙이는 속성

그림의 오른쪽은 트레이스에 붙이는 속성입니다. environment는 staging과 production처럼 어느 환경에서 생긴 기록인지를 나누고, version은 어느 배포 버전이 만든 기록인지를 나눕니다. "업데이트한 뒤로 느려졌다"는 말을 확인하려면 version으로 트레이스를 나눠 볼 수 있어야 합니다. tags와 metadata는 고른 도구, 도구 선택 확신도, 안전 응답 여부처럼 나중에 걸러 보고 싶은 값을 담는 칸입니다.

Langfuse가 작동하는 방식

Langfuse는 서비스 코드 안에서 기록을 만드는 SDK, 기록을 받아 저장하는 Langfuse 서버, 저장된 기록을 보여 주는 웹 화면으로 나뉩니다. 아래 그림은 메시지 하나의 기록이 SDK에서 화면까지 가는 길입니다.

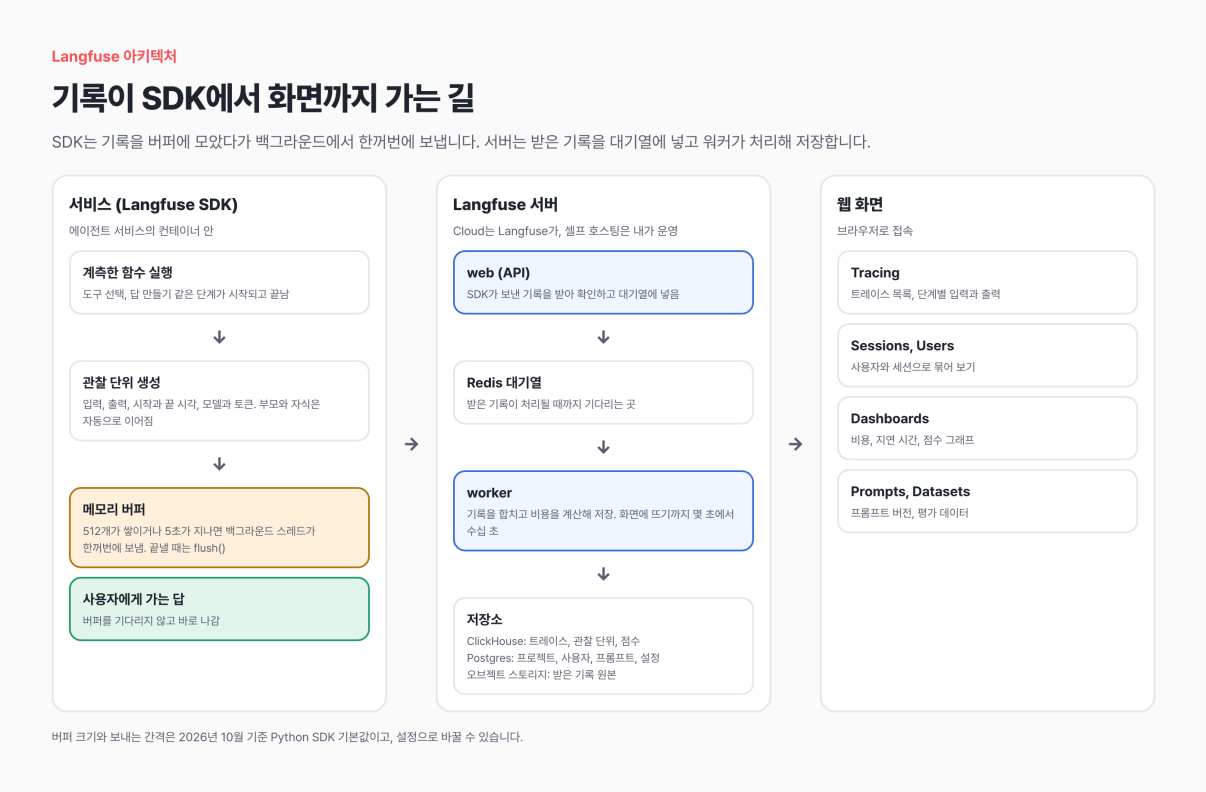


그림: Langfuse SDK가 기록을 모아 비동기로 보내고, 서버가 받아 저장한 뒤 화면에서 보는 흐름

계측한 함수가 실행되면 SDK가 관찰 단위를 만들고, 이 기록은 곧바로 서버로 가지 않고 메모리의 버퍼에 쌓입니다. SDK의 백그라운드 스레드가 기본값으로 기록이 512개 쌓이거나 5초가 지나면 한꺼번에 보냅니다(2026년 10월 기준 Python SDK 기본값). 비동기로 보내므로 사용자는 기록을 보내는 시간만큼 답을 늦게 받지 않고, Langfuse 서버가 잠시 응답하지 않아도 서비스의 답은 그대로 나갑니다. 대신 서버가 받은 기록을 대기열에 넣고 처리하므로 트레이스가 화면에 뜨기까지 몇 초에서 수십 초가 걸리고, 프로세스가 강제로 끝나면 버퍼에 남은 기록을 잃습니다. 짧게 돌고 끝나는 스크립트나 노트북 셀의 끝에서 `flush()` 를 불러 버퍼를 비우는 이유입니다.

지금의 Langfuse Python SDK는 프로그램의 트레이스, 지표, 로그를 남기고 보내는 방법을 정한 공개 표준인 OpenTelemetry(줄여서 OTel) 위에서 만들어져 있습니다. 위에서 본 자동 중첩도 이 표준의 컨텍스트 전파 덕분에 됩니다. 표준을 따르므로 LLM API 클라이언트나 LangChain처럼 OpenTelemetry 계측을 지원하는 라이브러리의 호출은 연동 설정만으로 기록되는데, 이것을 자동 계측(auto-instrumentation)이라고 합니다. 다만 자동 계측은

라이브러리가 아는 단위로만 기록해서, 모델 호출은 남지만 "도구 선택"이나 "기억 정리" 같은 단계 이름은 모릅니다.

Langfuse Cloud와 셀프 호스팅

Langfuse는 Langfuse 회사가 서버를 운영해 주는 Langfuse Cloud로 쓰거나, 공개된 코드로 내 서버에 직접 띄우는 셀프 호스팅(self-hosting)으로 씁니다. 셀프 호스팅하면 web과 worker 두 컨테이너에 Postgres, ClickHouse, Redis, 오브젝트 스토리지까지 함께 운영해야 하므로, 대화 데이터를 회사 밖으로 보낼 수 없는 규제가 있거나 트래픽이 아주 클 때 고릅니다. 이 과정에서는 설치 없이 바로 쓰는 Langfuse Cloud의 무료 플랜(Hobby)을 씁니다. 무료 플랜은 한 달 5만 단위(트레이스와 관찰 단위 등을 합친 수)까지 기록하고, 30일 지난 데이터는 볼 수 없으며, 프로젝트당 사용자는 2명입니다(2026년 10월 기준). SDK 코드는 두 방식에서 같고, 보낼 서버 주소(`LANGFUSE_BASE_URL`)와 키만 다릅니다.

객관식 퀴즈

Q. 평가용 메시지 20개를 보내는 스크립트를 내 컴퓨터에서 돌렸습니다. 스크립트는 오류 없이 바로 끝났는데, Langfuse 화면에는 트레이스가 12개만 보이고 몇 분을 기다려도 늘지 않습니다. 가장 먼저 의심할 것은 무엇인가요?

• 1. Langfuse 무료 플랜의 한 달 한도를 넘었다
• 2. SDK 버퍼에 남은 기록을 보내기 전에 프로세스가 끝났다. 스크립트 끝에서 `flush()`를 부르지 않았는지 본다
• 3. LLM 제공사가 요청 8개를 거절했다
• 4. Langfuse 서버가 트레이스를 30일 동안 보관하지 않았다**제출**

- •

  1. Langfuse 무료 플랜의 한 달 한도를 넘었다

- •

  2. SDK 버퍼에 남은 기록을 보내기 전에 프로세스가 끝났다. 스크립트 끝에서 `flush()`를 부르지 않았는지 본다

- •

  3. LLM 제공사가 요청 8개를 거절했다

- •

  4. Langfuse 서버가 트레이스를 30일 동안 보관하지 않았다

📚 Langfuse 참고 자료

- Langfuse: Data model: 트레이스, 관찰 단위, 세션, 점수가 어떻게 이어지는지 공식 문서로 볼 수 있습니다(영어).
- Langfuse: Python SDK overview: SDK 설치와 환경변수, 클라이언트 만들기, flush를 확인할 수 있습니다. 계측을 맡길 때 코딩 에이전트에게 근거로 주는 문서입니다.
- Langfuse: OpenTelemetry: Langfuse가 OpenTelemetry 표준을 어떻게 쓰는지와 다른 OpenTelemetry 계측을 받는 방법이 있습니다.
- Langfuse: Self-hosting: 셀프 호스팅의 구성 요소와 Docker Compose로 내 컴퓨터에 띄우는 방법을 볼 수 있습니다.

Chapter03

시작 상태 맞추기

이 교안의 실습은 아래 상태가 갖춰져 있어야 진행할 수 있습니다. 섹션 4~6은 내 컴퓨터에서 에이전트 서비스를 띄워 하고, 섹션 7부터 staging 서비스에 붙입니다. 비용과 토큰을 보려면 실제 LLM API 키가 있어야 합니다. 가짜 LLM(`LLM_PROVIDER=fake`)으로도 트레이스의 구조는 볼 수 있지만 토큰과 비용이 0으로 남아 늘어난 요금의 원인을 확인할 수 없습니다.

앞 실습을 이어서 하고 있다면 아래 확인 명령만 실행하고 넘어갑니다. 빠진 상태가 있으면 템플릿의 실습별 시작 상태 문서에서 "없으면" 칸의 항목을 표의 위 칸부터 차례로 실행합니다. 처음부터 모두 맞추면 한두 시간이 걸리므로, 빠진 항목이 많다면 수업 전에 맞춰 두거나 강사에게 먼저 알립니다.

필요한 상태	확인할 것	없으면
템플릿으로 만든 내 저장소와 가상환경, `.env` 에 실제 LLM API 키	가상환경을 켜고 `uvicorn app.main:app` 으로 띄운 화면에서 답이 옴	A의 3번, 저장소 README의 로컬 실행 준비
staging과 production 서비스, Actions 배포 워크플로	`gcloud run services list` 에 두 서비스, `deploy.yml` 에 `docker build`	A, B, C, D, E, F, G
staging과 production의 Supabase 연결	staging 화면 왼쪽 아래 "게스트로 사용 중"	I (연결하지 않았다면 사용자가 모두 `local-user` 하나로 기록되지만 실습은 진행할 수 있음)

직접 실행할 명령: 시작 상태 확인

```
python -c "import app.main" && echo ok
gcloud run services list --region us-central1
```

```
grep -n "docker build" .github/workflows/deploy.yml
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 의 표를 보고, 이 저장소와 지금 gcloud 프로젝트가 'Langfuse 관측' 실습을 시작할 상태인지 확인해 줘.
확인 명령만 실행하고 아무것도 만들거나 바꾸지 마. 빠진 항목이 있으면 문서의 어느 항목(A~I)을 실행해야 하는지 알려 줘.
```

Langfuse Cloud 프로젝트 만들기

Langfuse Cloud는 데이터를 둘 지역(리전)을 가입할 때 고릅니다. 에이전트 서비스가 미국(us-central1)의 Cloud Run에서 돌므로 미국 리전을 고르면 트레이스를 보낼 때 지연이 적습니다. 리전에 따라 접속 주소가 다르고(미국 `https://us.cloud.langfuse.com`, 유럽 `https://cloud.langfuse.com`), 이 주소를 SDK 설정에도 그대로 넣어야 합니다. 리전이 다르면 키가 맞아도 트레이스가 들어가지 않습니다.

직접 확인할 곳: Langfuse 프로젝트와 키 만들기

1. `https://us.cloud.langfuse.com` 에 가입합니다. GitHub 계정으로 가입할 수 있습니다.
2. 조직(Organization)과 프로젝트를 만듭니다. 프로젝트 이름은 `friend-agent` 처럼 짓고, staging과 production은 프로젝트를 나누지 않고 environment 속성으로 나눕니다.
3. 프로젝트 > Settings > API Keys 에서 키를 새로 만듭니다. 공개키(`pk-lf-` 로 시작), 비밀키(`sk-lf-` 로 시작), 접속 주소(Host) 세 가지가 나옵니다.
4. 비밀키는 이 화면을 닫으면 다시 볼 수 없습니다. 바로 저장소 루트의 `.env` 에 넣습니다. 키 값은 채팅이나 명령, 코드에 붙여 넣지 않습니다.

`.env` 에는 세 줄을 더합니다. 이름은 Langfuse SDK가 읽는 환경변수 이름 그대로 씁니다.

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_BASE_URL=https://us.cloud.langfuse.com
```

공개키는 이름과 달리 혼자서는 아무것도 할 수 없는 식별자이고, 비밀키가 있어야 기록을 보낼 수 있습니다. 비밀키가 드러나면 누구든 내 프로젝트에 가짜 트레이스를 쌓을 수 있으므로, 배포한 서비스에서는 LLM API 키처럼 Secret Manager에 둡니다(섹션 7). `.env.example` 에도 같은 이름을 값 없이 적어 두면, 저장소를 받은 사람이 어떤 값이 필요한지 알 수 있습니다.

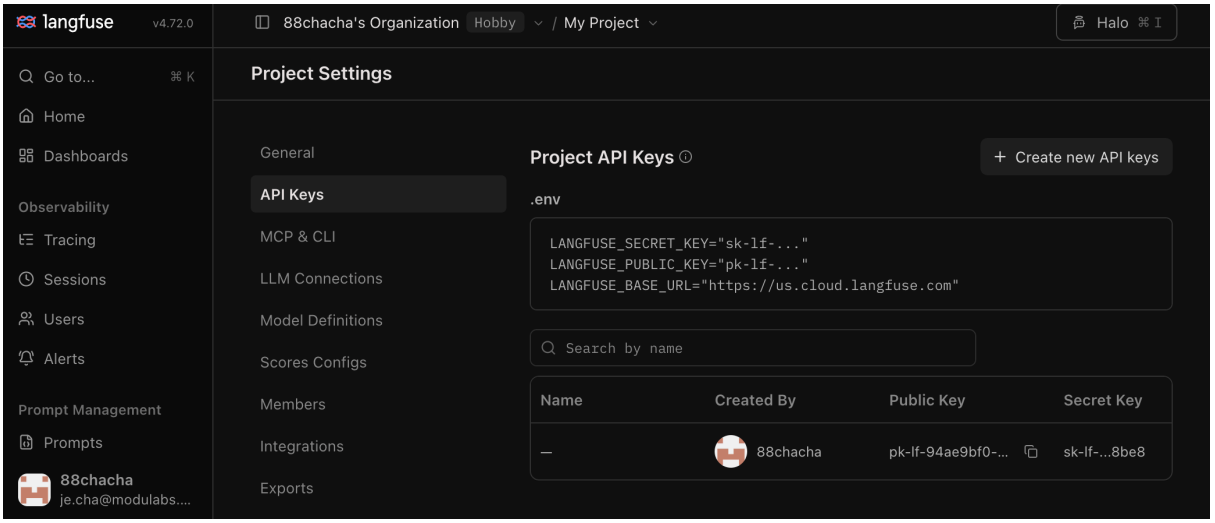


그림: Langfuse > 프로젝트 > Settings > API Keys: 공개키와 접속 주소가 보이고 비밀키는 가려진 키 목록

객관식 퀴즈

Q. Langfuse 키를 넣고 계측까지 했는데 트레이스가 하나도 들어오지 않습니다. 오류 메시지도 없습니다. 가장 먼저 확인할 것은 무엇인가요?

- 1. Cloud Run 인스턴스 수
- 2. `LANGFUSE_BASE_URL` 이 가입할 때 고른 리전의 주소와 같은지
- 3. LLM API 키의 잔액
- 4. Supabase 익명 로그인 설정

- •

  1. Cloud Run 인스턴스 수

- •

  2. `LANGFUSE_BASE_URL` 이 가입할 때 고른 리전의 주소와 같은지

- •

  3. LLM API 키의 잔액

- •

  4. Supabase 익명 로그인 설정

Chapter04

엉뚱한 날씨 답

동료는 "어제 저녁에 도쿄 날씨를 물었는데 엉뚱한 답이 나왔어요"라고 했습니다. 서버 로그에서 어제 저녁을 찾아보면 `POST /api/chat 200` 여러 줄 사이에 `get_weather` 를 골랐다는 도구 선택 로그가 보이지만, 그 줄이 동료의 메시지였는지, 도구가 무엇을 돌려줬는지, 모델이 무엇을 보고 그 답을 만들었는지는 남아 있지 않습니다. 이 문제에 답하려면 그 메시지 하나에서 일어난 일을 한 묶음으로, 단계마다 입력과 출력까지 봐야 합니다.

Langfuse에서는 그 메시지의 트레이스를 열어 단계별 입력과 출력을 보면 됩니다. 그런데 트레이스에 어떤 단계가 보일지는 계측을 어떻게 했느냐로 정해집니다. 이 섹션에서는 한 턴의 트레이스가 어떤 모양이어야 엉뚱한 날씨 답의 원인을 찾을 수 있는지 먼저 보고, 그 모양대로 계측 구조를 정합니다.

움직여 보는 한 턴의 트레이스

섹션 2의 데이터 모델이 에이전트 서비스의 메시지 하나에서 어떻게 채워지는지 봅니다. 아래 시각화에서 탭으로 메시지를 고르고 "메시지 보내기"를 누르면 단계가 차례로 기록되고, 줄을 누르면 그 단계의 입력과 출력이 나옵니다. ③ 탭이 동료가 겪은 도쿄 날씨와 같은 경우입니다. 도구 실행이 실패했는데 답이 날씨를 지어낸 트레이스에서, 어느 줄을 열어야 그 사실이 보이는지 찾아봅니다. ① 탭에서는 모델 호출이 세 번 일어나고, 기억 정리는 사용자가 답을 다 받은 뒤(파란 선)에 이어지지만 비용은 똑같이 든다는 점을 봅니다. 느린 답과 늘어난 요금을 볼 때 다시 쓰는 사실입니다. ④ 탭에서는 안전장치가 모델을 부르지 않고 답을 정해 비용이 0인 트레이스를 봅니다. "가짜 LLM"을 누르면 토큰과 비용이 모두 0으로 바뀝니다.

객관식 퀴즈

Q. 동료가 도쿄 날씨를 물었는데 엉뚱한 날씨를 들었다고 합니다. 서버 로그에는 그 시각의 `POST /api/chat 200` 과 `get_weather` 를 골랐다는 도구 선택 로그가 남아 있습니다. 트레이스가 있다면 가장 먼저 확인할 것은 무엇인가요?

• 1. Cloud Run 인스턴스 수가 부족했는지
• 2. 그 트레이스의 도구 실행 출력과 답 만들기의 입력, 출력을 나란히 보고 도구 결과를 모델이 어떻게 다뤘는지
• 3. 응답 코드가 200이었는지
• 4. 같은 시각 다른 사용자의 요청 수**제출**

- •

  1. Cloud Run 인스턴스 수가 부족했는지

- •

2. 그 트레이스의 도구 실행 출력과 답 만들기의 입력, 출력을 나란히 보고 도구 결과를 모델이 어떻게 다뤘는지

- •

3. 응답 코드가 200이었는지

- •

4. 같은 시각 다른 사용자의 요청 수

계측 구조를 먼저 정하는 이유

계측은 코딩 에이전트에게 맡기지만, 무엇을 어떤 단위로 남길지는 먼저 정해야 합니다. "Langfuse 붙여 줘"라고만 하면 에이전트는 두 방향 가운데 하나로 가기 쉽습니다. 하나는 LLM API 호출만 자동 계측으로 남기는 것입니다. 이러면 모델 호출 세 개가 나란히 보이지만 어느 것이 도구 선택이고 어느 것이 답 만들기인지 이름이 없고, 모델을 부르지 않는 도구 실행은 아예 남지 않아서 도쿄 날씨 질문에 도구가 무엇을 돌려줬는지 볼 수 없습니다. 다른 하나는 눈에 띄는 함수마다 기록을 붙이는 것입니다. 이러면 한 턴에 관찰 단위가 수십 개 생겨 봐야 할 단계를 찾기 어렵고, 무료 플랜의 한 달 한도도 빨리 씁니다.

어느 쪽이 맞는지는 코드만 봐서는 정할 수 없습니다. 이 트레이스로 무엇을 확인하려는지를 아는 사람이 정해야 하고, 그 기준은 섹션 1에서 동료들이 한 말입니다. 엉뚱한 날씨 답의 원인을 찾으려면 도구 선택, 도구 실행, 답 만들기가 따로 보여야 하고, 늘어난 요금을 나눠 보려면 모델을 부르는 단계마다 토큰이 남아야 하며, 잊어버린 이사 이야기를 따라가려면 기억 정리의 출력이 남아야 합니다. 아래에서 단계 나누기, 종류 정하기, 남길 것 정하기, 이름 정하기 순서로 계측 구조를 정합니다.

한 턴을 단계로 나누기

템플릿의 `app/agent.py` 를 열면 맨 위 설명에 한 턴의 흐름이 ①부터 ⑤까지 적혀 있고, `run_turn` 함수도 같은 순서로 짜여 있습니다. 안전장치, 도구 선택, 도구 실행, 답 만들기, 기억 정리입니다. 이 다섯 단계를 관찰 단위로 삼습니다. 더 잘게 나누지 않는 기준은 고치는 단위입니다. 엉뚱한 답의 원인이 도구 선택이면 도구 선택 프롬프트나 규칙을, 도구 실행이면 도구 코드를, 답 만들기면 답 만들기 프롬프트를 고치므로, 고칠 곳이 다른 단계는 따로 보여야 합니다. 반대로 답 만들기 안에서 시스템 프롬프트를 조립하는 함수와 모델을 부르는 함수처럼 고칠 곳이 같은 함수들은 한 단계로 묶습니다.

도구 실행은 도구마다 하나씩 남깁니다. 한 메시지에 도구를 두 개까지 고를 수 있고, 날씨와 공휴일처럼 도구마다 실패하는 이유가 다르기 때문입니다. 기억 정리 뒤에는 요약되지 않은 메시지가 많을 때만 대화를 요약하는 단계(`maybe_summarize`)가 같은 요청 안에서 이어지므로, 이것도 관찰 단위 하나로 둡니다.

단계들을 묶는 트레이스도 정합니다. `run_turn` 이 한 번 실행되는 것이 메시지 하나이므로, 트레이스는 `run_turn` 단위로 만듭니다. 화면을 열 때 먼저 말을 거는 `greet` 는 `/api/greeting` 이라는 다른 요청에서 실행되므로 트레이스를 따로 만듭니다.

단계마다 종류 정하기

관찰 단위의 종류는 그 단계가 모델을 부르는지로 정합니다. 모델을 부르는 단계를 generation으로 남겨야 Langfuse가 모델 이름과 토큰 수로 비용을 계산하고, 모델을 부르지 않는 단계는 span으로 남깁니다. 단계별 비용을 나눠 보려면 이 구분이 맞아야 합니다. 아래 표의 파일을 직접 열어 모델을 부르는지 확인해 봅니다.

단계	코드에서 확인할 곳	모델을 부르나	종류
안전장치	`app/safety.py` 의 `is_crisis`	부르지 않음. 위기 표현 목록과 비교하는 코드 규칙	span
도구 선택	`app/router.py` 의 `choose`	실제 LLM이면 부름. 가짜 LLM이거나 모델 응답을 읽지 못하면 키워드 규칙이 대신 고름	generation
도구 실행	`app/tools.py` 의 `run_tool`	부르지 않음. 날씨 API 같은 바깥 서비스를 부름	span
답 만들기	`app/agent.py` 에서 `llm.chat_stream` 을 부르는 곳	부름. 답을 스트리밍으로 받음	generation
기억 정리	`app/memory.py` 의 `update_memories`	실제 LLM이면 부름	generation
대화 요약	`app/memory.py` 의 `maybe_summarize`	요약할 때만 부름	generation

도구 선택은 규칙이 고를 때 모델을 부르지 않으므로 그때는 토큰이 0으로 남습니다. 모델이 정했는지 규칙이 정했는지(`decided_by`)를 함께 남겨 두면, 토큰이 0인 트레이스를 보고 계측이 빠진 것인지 규칙이 정한 것인지 헷갈리지 않습니다.

단계마다 남길 것 정하기

단계마다 무엇을 남길지는 "이 단계가 틀렸다는 것을 무엇을 보고 알 수 있나"로 정합니다. 대부분의 단계는 입력과 출력이면 충분합니다. 도구 선택은 고른 도구와 인자에 더해 확신도와 누가 정했는지를 남겨야, 도구를 쓰지 않고 되묻기로 넘어간 이유를 알 수 있습니다. 도구 실행은 성공 여부를 남겨야 실패한 도구만 걸러 볼 수 있습니다.

generation에는 모델 이름과 입력, 출력 토큰 수를 꼭 남깁니다. 이 두 값이 없으면 비용이 계산되지 않습니다. 답 만들기에는 첫 글자가 나온 시각을 더합니다. 답을 스트리밍으로 보내므로 사용자가 기다린 시간은 답이 끝난 시각이 아니라 첫 글자가 나온 시각까지이고, 느린 답을 볼 때 이 값이 필요합니다.

이름 정하기와 계측 구조 정리

관찰 단위 이름은 Langfuse 화면에서 거르고 묶는 기준이 됩니다. 같은 단계는 어느 트레이스에서나 같은 이름이어야 `reply` 만 모아 지연 시간을 보거나 `memory-update` 의 비용만 합칠 수 있습니다. 함수 이름을 그대로 쓰면 코드를 고칠 때 이름도 바뀌므로, 화면에서 읽기 쉬운 이름을 따로 정해 둡니다. 지금까지 정한 것을 모으면 아래 표가 되고, 이 표가 섹션 5에서 코딩 에이전트에게 건넬 계측 구조입니다.

단계	관찰 단위 이름	종류	남길 것	쓰는 곳
한 턴 전체	`chat-turn`	트레이스	사용자 메시지, 최종 답, 메시지 ID	전부
안전장치	`safety`	span	위기 표현 감지 여부	엉뚱한 답
도구 선택	`choose-tools`	generation	모델 입력과 출력, 고른 도구, 확신도, 누가 정했는지(모델, 규칙)	엉뚱한 답, 느린 답, 요금
도구 실행	`tool:<도구 이름>`	span	인자, 결과, 성공 여부	엉뚱한 답, 느린 답
답 만들기	`reply`	generation	모델에 보낸 메시지, 답, 토큰, 첫 글자가 나온 시각	엉뚱한 답, 느린 답, 요금, 기억
기억 정리	`memory-update`	generation	추가, 수정, 삭제한 기억	요금, 기억
대화 요약	`memory-summary`	generation	요약에 합친 메시지 수, 새 요약	요금
먼저 말 걸기	`greeting`	트레이스	인사말	요금

대화 요약은 요약되지 않은 메시지가 40개를 넘을 때만, 먼저 말 걸기는 대화가 처음이거나 6시간 넘게 쉬었거나 전할 약속이 있을 때만 모델을 부릅니다. 자주 일어나지 않는 호출이라도 따로 이름을 붙여 두면, 비용이 어디서 나왔는지 나눠 볼 때 빠지지 않습니다.

객관식 퀴즈

Q. 코딩 에이전트가 계측을 넣으면서 기억 정리(`memory-update`)를 generation이 아닌 span으로 남겼습니다. 트레이스에서 볼 수 없게 되는 것은 무엇인가요?

- 1. 기억 정리가 추가한 기억
- 2. 기억 정리 호출의 모델 이름, 토큰 수, 비용
- 3. 기억 정리가 걸린 시간
- 4. 기억 정리가 한 턴 안에서 실행된 순서**제출**

- •

  1. 기억 정리가 추가한 기억

- •

  2. 기억 정리 호출의 모델 이름, 토큰 수, 비용

- •

  3. 기억 정리가 걸린 시간

- •

  4. 기억 정리가 한 턴 안에서 실행된 순서

속성과 개인정보

관찰 단위를 정했으면, 트레이스에 붙일 속성과 남기지 않을 것을 정합니다. 속성은 섹션 7에서 이사 이야기와 업데이트 뒤 느려진 답을 확인할 때 붙이지만, 무엇을 붙일지는 지금 함께 정해 둡니다. 아래 그림은 각 속성의 값을 서비스의 어디에서 가져올지와 정할 것을 정리한 것입니다.



그림: 트레이스 속성의 값을 서비스의 어디에서 가져오고 무엇을 정해야 하는지

세션은 Langfuse가 정해 주지 않습니다. 에이전트 서비스에는 대화방이 따로 없고 사용자 한 명의 대화가 계속 이어지므로, "사용자 ID와 한국 날짜를 합친 값"처럼 하루치 대화를 한 세션으로 보는 식으로 정합니다. 그러면 세션 화면에서 한 사람이 하루 동안 나눈 대화를 순서대로 볼 수 있습니다.

가장 신중하게 정할 것은 입력과 출력입니다. 트레이스에는 사용자가 보낸 메시지와 모델의 답, 기억 메모가 그대로 올라갑니다. 템플릿의 작업 규칙(`AGENTS.md`)은 대화와 기억을 개인정보로 보고 서버 로그에 원문을 남기지 않게 하는데, 원문이 없으면 도쿄 날씨처럼 답이 왜 이상했는지 읽을 수 없습니다. 그래서 실무에서는 접근할 수 있는 사람이 정해진 관측 도구에는 원문을 남기되, 전화번호나 이메일, 카드 번호 같은 패턴은 보내기 전에 가리고(마스킹), 보관 기간을 정하고, 개인정보처리방침에 외부 관측 도구로 보낸다는 사실을 적습니다. Langfuse SDK에는 보내기 전에 값을 바꾸는 마스킹 기능이 있습니다. 이 결정을 요청 틀에 적어 두지 않으면, 코딩 에이전트는 작업 규칙과 부딪혀 원문을 아예 빼거나, 반대로 아무것도 가리지 않고 보낼 수 있습니다.

SDK 버전 주의

Langfuse Python SDK는 지금 v5가 최신이고(2026년 10월 기준), 웹에 있는 예제 상당수는 v2, v3 시절 문법이라 그대로 쓰면 동작하지 않습니다. 코딩 에이전트도 학습한 시점의 옛 문법을 쓰기 쉽습니다. 그래서 요청할 때 "공식 문서의 최신 SDK 문서를 먼저 읽고, 어느 문서를 근거로 했는지 알려 달라"고 함께 적고, 에이전트가 쓴 코드의 import 줄과 함수 이름이 공식 문서와 같은지 확인합니다.

객관식 퀴즈

Q. 에이전트 서비스의 트레이스에 사용자 메시지 원문을 남기기로 했습니다. 함께 정해야 할 것으로 가장 알맞은 것은 무엇인가요?

• 1. 원문은 남기므로 더 정할 것이 없다
• 2. 전화번호, 이메일 같은 패턴을 보내기 전에 가리고, 누가 볼 수 있는지와 보관 기간, 개인정보처리방침의 안내를 정한다
• 3. 서버 로그에도 같은 원문을 남겨 두 곳에서 볼 수 있게 한다
• 4. 원문 대신 메시지 길이만 남긴다

- •

  1. 원문은 남기므로 더 정할 것이 없다

- •

  2. 전화번호, 이메일 같은 패턴을 보내기 전에 가리고, 누가 볼 수 있는지와 보관 기간, 개인정보처리방침의 안내를 정한다

- •

  3. 서버 로그에도 같은 원문을 남겨 두 곳에서 볼 수 있게 한다

- •

  4. 원문 대신 메시지 길이만 남긴다

Chapter05

요청 틀 쓰는 법

섹션 4에서 정한 계측 구조를 코딩 에이전트에게 맡깁니다. 이 교안의 요청 틀은 Claude Code 같은 코딩 에이전트에게 보낼 메시지의 초안입니다. 저장소 폴더에서 코딩 에이전트를 열고, 아래 요청 틀을 복사해 붙여 넣은 뒤 대괄호(`[ ]`) 안을 내가 정한 값으로 바꿔 보냅니다. 대괄호로 비워 둔 곳은 서비스를 운영하는 사람이 정해야 하는 값이라, 에이전트에게 정해 달라고 넘기지 않습니다.

한 번에 다 하지 않고, 이 섹션에서는 관찰 단위와 입력, 출력만 넣습니다. 사용자, 세션, 환경, 버전은 섹션 7에서, 점수는 섹션 8에서 더합니다. 단계를 나눠 맡겨야 무엇이 잘못됐을 때 어느 변경 때문인지 찾기 쉽습니다.

요청 틀(빈칸은 직접 채운다):

```
작업 브랜치를 만들고 에이전트 서비스에 Langfuse 계측을 넣어 줘.
- 먼저 Langfuse 공식 문서(Python SDK 최신 버전)를 읽고, 근거로 한 문서 주소를 알려 줘. 옛 버전 문법은 쓰지 마.
- requirements.txt 에 langfuse 를 추가해 줘. 버전은 [  ] 로 고정.
- 관찰 단위는 아래 표대로. 이름도 그대로 써 줘.
  chat-turn(트레이스), safety(span), choose-tools(generation), tool:<도구 이름>(span), reply(generation),
  memory-update(generation), memory-summary(generation), greeting(트레이스)
- generation 에는 모델 이름과 입력, 출력 토큰 수를 남겨 줘. 가짜 LLM 일 때는 모델 이름을 fake 로.
- choose-tools 에는 고른 도구, 확신도, 누가 정했는지(decided_by)를, tool 에는 성공 여부를 남겨 줘.
- reply 에는 답의 첫 글자가 나온 시각도 남겨 줘.
- 입력과 출력 원문은 [남긴다 / 남기지 않는다]. 남긴다면 보내기 전에 [전화번호, 이메일, 카드 번호] 패턴은 가려 줘.
- LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY 가 없으면 계측을 끄고 앱은 그대로 돌게 해 줘.
- 기존 테스트(pytest)가 모두 통과해야 해. 서버 로그에 대화 원문을 남기지 않는 규칙은 그대로 지켜 줘.
- 바꾸는 파일과 이유를 먼저 설명하고, 바꾼 뒤 테스트 결과를 보여 줘.
```

요청 틀의 줄마다 들어간 이유

요청 틀의 줄은 섹션 4에서 정한 것과, 계측을 넣다가 서비스가 망가지지 않게 하는 조건으로 나뉩니다. 빈칸을 채우기 전에 줄마다 왜 들어갔는지 읽어 봅니다.

줄	들어간 이유	빈칸에 넣을 값
공식 문서를 먼저 읽고 근거 주소를 알려 달라	에이전트가 옛 SDK 문법을 쓰는 것을 막고, 내가 근거를 확인할 수 있게 함	없음
버전 고정	배포할 때마다 다른 버전이 설치되면 어제 되던 계측이 오늘 깨질 수 있음	에이전트가 근거로 든 문서의 버전 번호. 모르면 "지금 최신 버전을 확인해 그 번호로"라고 적음
관찰 단위와 이름	섹션 4의 표. 이름이 다르면 뒤 섹션에서 화면을 확인하는 단계와 맞지 않음	없음
generation의 모델 이름과 토큰, 도구 선택과 도구 실행에 남길 값, 첫 글자 시각	섹션 4에서 단계마다 남기기로 정한 것	없음
원문을 남길지와 가릴 패턴	섹션 4의 개인정보 결정	이 실습에서는 "남긴다"와 전화번호, 이메일, 카드 번호
키가 없으면 계측을 끔	CI 테스트나 키를 넣지 않은 동료의 컴퓨터에서 앱이 멈추지 않게 함	없음
기존 테스트와 로그 규칙	템플릿의 작업 규칙(`AGENTS.md`)을 지키고, 계측 때문에 다른 기능이 깨지지 않았는지 확인함	없음
먼저 설명하고 테스트 결과를 보여 달라	승인하기 전에 무엇을 바꿀지 읽을 수 있게 함	없음

▼ 빈칸을 채운 요청의 예

버전 줄은 "버전은 지금 PyPI의 최신 버전을 확인해 그 번호로 고정", 원문 줄은 "입력과 출력 원문은 남긴다. 보내기 전에 전화번호, 이메일, 카드 번호 패턴은 가려 줘"처럼 채웁니다. 나머지 줄은 그대로 둡니다. 원문을 남기지 않기로 정했다면 가릴 패턴 대신 "입력과 출력에는 글자 수만 남겨 줘"라고 적을 수 있지만, 그러면 이 섹션 뒤에서 엉뚱한 답의 원인을 읽을 수 없습니다.

직접 확인할 곳: 코딩 에이전트에게 계측 맡기기

1. 터미널에서 템플릿으로 만든 내 저장소 폴더로 이동해 코딩 에이전트를 엽니다.
2. 빈칸을 채운 요청을 붙여 넣어 보냅니다.

3. 에이전트가 근거로 읽은 문서 주소와 바꿀 파일, 이유를 먼저 보여 줍니다. 아래 문단의 다섯 가지를 확인한 뒤 승인합니다. 이해되지 않는 변경이 있으면 승인하기 전에 그 줄이 왜 필요한지 묻습니다.
4. 에이전트가 파일을 고치고 `pytest` 결과를 보여 줍니다. 실패한 테스트가 있으면 바로 고치게 하지 말고, 원인을 먼저 설명하게 합니다.
5. 바뀐 파일 목록에 `.env` 가 들어 있지 않은지 확인합니다.

승인하기 전에 볼 다섯 가지는 import 줄과 함수 이름이 근거로 든 공식 문서와 같은지, 섹션 4 표의 관찰 단위 이름이 그대로인지, 키가 없을 때 계측을 끄는 분기가 있는지, 서버 로그(`log(...)`)에 원문을 넣는 줄이 새로 생기지 않았는지, 키 값이 코드나 테스트 파일에 들어가지 않았는지입니다. 템플릿의 LLM 호출은 OpenAI 호환, Claude, 가짜 LLM 세 가지 클래스(`app/llm.py`)로 나뉘어 있어서, 에이전트가 한 제공사의 자동 계측 통합만 쓰면 다른 제공사로 바꿨을 때 기록이 빠질 수 있습니다. 세 클래스가 모두 같은 방식으로 기록되는지도 물어봅니다.

직접 실행할 명령: 내 컴퓨터에서 띄우고 테스트

```
# 저장소 루트, 가상환경을 켠 터미널에서
pip install -r requirements.txt
pytest -q
uvicorn app.main:app --reload --port 8080
```

직접 확인할 곳: 트레이스가 들어오는지 확인

1. `http://localhost:8080` 에서 캐릭터를 저장하고 평범한 잡담을 두세 개 보냅니다.
2. Langfuse > Tracing > Traces 에서 보낸 메시지 수만큼 `chat-turn` 트레이스가 생겼는지 봅니다. 처음 대화를 시작했다면 먼저 말 걸기의 `greeting` 트레이스도 하나 보입니다.
3. 트레이스 하나를 열어 왼쪽 나무 구조에 `safety`, `choose-tools`, `reply`, `memory-update` 가 차례로 있는지 봅니다. 잡담에는 도구를 쓰지 않으므로 `tool:…` 은 없습니다.
4. `reply` 를 눌러 모델 이름, 입력과 출력 토큰, 비용이 나오는지 봅니다.

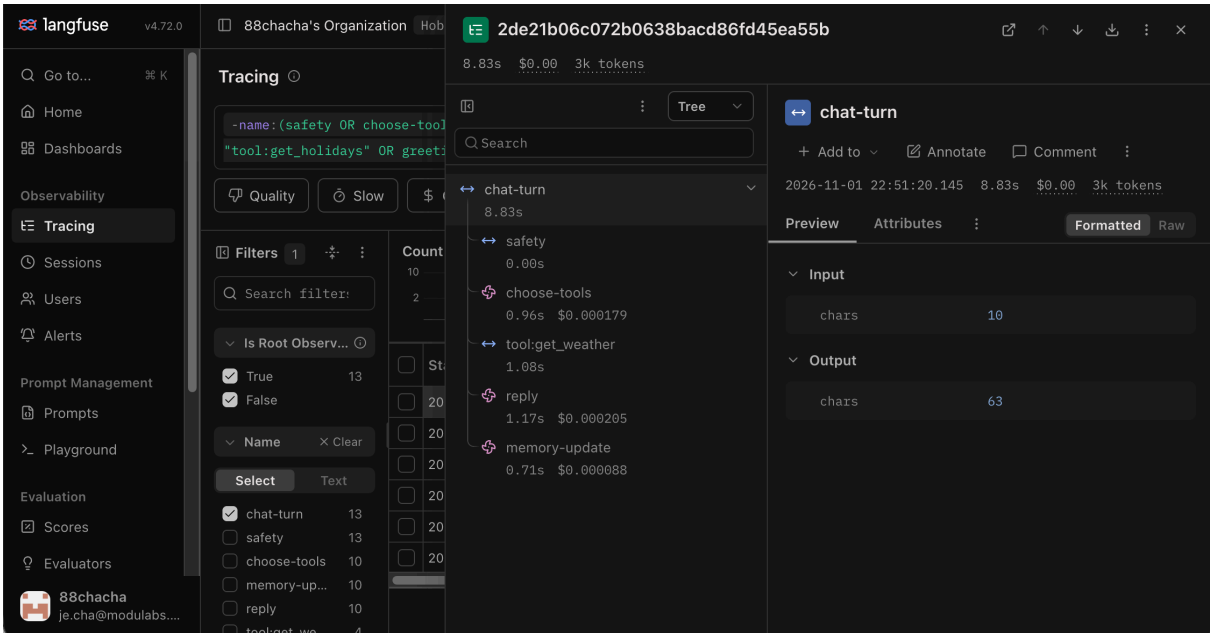


그림: Langfuse > Tracing > Traces > chat-turn 트레이스 하나: 왼쪽에 단계 나무 구조, 오른쪽에 reply 의 입력과 출력, 토큰과 비용이 보임

▼ 트레이스가 들어오지 않을 때

먼저 `.env` 의 세 값 이름이 정확한지, 서버를 다시 띄웠는지 봅니다. 그다음 서버 터미널에 Langfuse 관련 경고가 있는지 봅니다. 섹션 2에서 본 것처럼 트레이스는 버퍼에 모았다가 따로 보내므로, 화면에 나타나기까지 몇 초에서 1분쯤 걸릴 수 있습니다. 1분이 지나도 없다면 코딩 에이전트에게 "Langfuse 클라이언트가 인증에 성공했는지 확인하는 방법을 공식 문서에서 찾아 알려 줘"라고 물어봅니다.

도쿄 날씨 질문 다시 만들고 거슬러 올라가기

트레이스가 들어오는 것을 확인했으면 동료가 겪은 도쿄 날씨 상황을 내 컴퓨터에서 다시 만듭니다. 에이전트 서비스의 날씨 도구는 정해 둔 국내 도시만 지원하므로, "도쿄 날씨 어때?"를 보내면 동료가 겪은 상황과 같은 트레이스가 생깁니다. 문제를 찾을 때는 답만 보지 않고, 답을 만들기 전에 모델이 무엇을 받았는지부터 거슬러 올라갑니다.

직접 확인할 곳: 거슬러 올라가며 읽기

1. "도쿄 날씨 어때?"를 보내고 화면에 나온 답을 적어 둡니다. 비교할 수 있게 "내일 서울 비 와?"도 하나 보냅니다.
2. 도쿄 트레이스를 열고 `reply` 의 출력에서 사용자가 본 답을 확인합니다.
3. `reply` 의 입력에서 시스템 프롬프트의 `[도구 결과]` 부분을 찾습니다. 도구가 실패했다는 사실이 모델에게 전달되었는지 봅니다.

4. `tool:get_weather` 의 출력에서 도구가 실제로 무엇을 돌려줬는지, 성공 여부가 무엇인지 봅니다.
5. `choose-tools` 의 출력에서 모델이 어떤 도구를 어떤 인자로 골랐는지, 확신도가 얼마였는지, 모델이 정했는지 규칙이 정했는지 봅니다.
6. 서울 트레이스의 같은 자리와 나란히 놓고, 도쿄 트레이스가 처음 달라진 단계를 한 문장으로 적습니다.

결과 해석: 처음 어긋난 단계

도쿄 트레이스에서는 `tool:get_weather` 가 지원하지 않는 지역이라는 실패 결과를 돌려주고, 그 문장이 `reply` 입력의 `[도구 결과]` 에 그대로 들어갑니다. 여기까지는 서비스가 정한 대로 움직인 것이므로, 답이 엉뚱했다면 그다음 단계를 봐야 합니다. 모델이 "도쿄 날씨는 아직 알려 줄 수 없어요"처럼 답했다면 이번 실행에서는 문제가 재현되지 않은 것이고, 그럴듯한 날씨를 지어냈다면 도구 결과를 받고도 무시한 것입니다. LLM은 같은 입력에도 답이 달라질 수 있으므로 몇 번 더 보내 보면 두 경우가 섞여 나오기도 합니다. 실행마다 결과가 다를 수 있다는 것 자체가 섹션 2에서 본 LLM 서비스의 특징입니다.

같은 "엉뚱한 답"이라도 처음 어긋난 단계에 따라 고칠 곳이 다릅니다. 도구 선택이 틀렸다면 도구 선택 프롬프트나 규칙을, 도구가 실패했다면 도구 쪽을, 도구 결과는 맞는데 답이 틀렸다면 답 만들기 프롬프트를 고쳐야 합니다. 도쿄의 경우 실패한 도구 결과를 어떻게 말할지 답 만들기 프롬프트에 정해 두지 않았다면, 고칠 곳은 답 만들기 프롬프트입니다. 이렇게 처음 어긋난 단계를 찾는 습관이 평가의 출발점이 됩니다. 실패한 트레이스를 모아 유형을 나누면 어떤 단계를 먼저 고쳐야 하는지 숫자로 보이기 때문입니다.

주관식 퀴즈

Q. [이해 점검] 내 컴퓨터에서 만든 도쿄 트레이스에서 처음 어긋난 단계는 어디였나요? 문제가 재현되지 않았다면 그렇다고 적고, 재현되었다면 그 단계를 고른 근거(어느 관찰 단위의 어떤 입력이나 출력)와 무엇을 고쳐야 할지 적어 보세요.

Chapter06

가끔 느린 답

동료는 "답이 가끔 한참 있다가 와요"라고만 했고, 언제였는지는 기억하지 못합니다. Cloud Run 콘솔의 요청 지연 시간 그래프에서도 느린 요청이 있었다는 것은 보이지만, 답을 스트리밍으로 보내므로 요청이 열려 있던 전체 시간이 찍힐 뿐 어느 단계가 늦었는지는 보이지

않습니다. Langfuse의 트레이스 목록은 지연 시간(Latency) 열로 정렬할 수 있고, 트레이스를 열면 관찰 단위마다 시작과 끝이 시간 막대로 보입니다. 섹션 5에서 계측한 그대로 쓸 수 있으므로 코드를 더 바꿀 필요는 없습니다.

직접 확인할 곳: 느린 트레이스 찾기

1. 섹션 5의 메시지에 더해, 도구를 쓰는 메시지("내일 부산 미세먼지 어때?", "다음 공휴일이 언제야?")와 도구 없는 잡담을 두세 개씩 보냅니다.
2. Tracing > Traces 에서 Latency 열로 정렬해 가장 느린 트레이스를 엽니다.
3. 시간 막대에서 `choose-tools`, `tool:…`, `reply`, `memory-update` 가 각각 차지한 시간을 적습니다.
4. `reply` 에서 첫 글자가 나오기까지 걸린 시간(Time to first token)을 봅니다. 사용자가 메시지를 보낸 뒤 화면에 첫 글자가 보이기까지 기다린 시간은 `reply` 가 시작되기 전의 단계와 이 시간을 더한 것입니다.
5. 가장 빠른 트레이스도 열어 같은 값을 적고, 두 트레이스가 어느 단계에서 차이가 났는지 봅니다.

결과 해석: 트레이스의 지연 시간과 사용자가 기다린 시간

트레이스 목록의 Latency는 사용자가 기다린 시간과 같지 않습니다. 기억 정리는 답의 글자가 화면에 다 나온 뒤에 같은 요청 안에서 돌기 때문에 트레이스 전체 시간에는 들어가지만, 사용자는 그 시간을 기다리지 않습니다. 사용자가 "한참 있다가 온다"고 느끼는 시간은 메시지를 보낸 뒤 첫 글자가 나올 때까지이고, 여기에는 도구 선택 호출, 도구 실행, 답의 첫 토큰까지가 들어갑니다. 그래서 느린 트레이스를 열면 `memory-update` 의 막대가 길어도 먼저 `reply` 앞쪽을 봅니다.

도구를 쓰는 메시지는 바깥 날씨 API를 부르는 시간만큼 첫 글자가 늦게 나오고, 도구 선택도 모델 호출이라 그 응답이 늦으면 전체가 밀립니다. 동료가 말한 "가끔"은 대개 이런 메시지였을 가능성이 높지만, 같은 단계라도 LLM 제공사의 응답 속도가 시간대에 따라 달라지므로 한두 개의 트레이스로 단정하지 않습니다. 느린 단계가 바깥 API라면 결과를 잠시 저장해 두고 다시 쓰는 방법을, 도구 선택이라면 키워드로 정할 수 있는 메시지는 규칙으로 정하는 방법을 생각해 볼 수 있습니다.

메시지 수보다 많은 요금

LLM 제공사의 청구 화면에는 날짜별 금액과 토큰 합계만 있고, 어느 기능이 쓴 것인지는 나오지 않습니다. 메시지 수에 모델 호출 한 번의 가격을 곱해 예상한 금액보다 청구서가 훨씬

많다면, 메시지 하나가 실제로 모델을 몇 번 부르고 각 호출이 얼마를 쓰는지를 나눠 봐야 합니다. Langfuse는 generation마다 모델 이름과 토큰 수로 비용을 계산하므로, 트레이스 하나 안에서 단계별로, 그리고 하루 단위로 비용을 나눠 볼 수 있습니다.

직접 확인할 곳: 비용을 나눠 보기

1. Tracing > Traces 에서 Cost 열로 정렬해 가장 비싼 트레이스와 가장 싼 트레이스를 엽니다.
2. 두 트레이스에서 `choose-tools`, `reply`, `memory-update` 의 입력 토큰, 출력 토큰, 비용을 적습니다.
3. 트레이스 목록에서 `chat-turn` 이 아닌 트레이스(`greeting`)가 몇 개인지, 모델을 불렀는지 봅니다.
4. Dashboards 에서 오늘의 트레이스 수와 총비용, 모델별 비용을 보고, 총비용을 보낸 메시지 수로 나눠 메시지 한 건의 평균 비용을 계산해 둡니다.

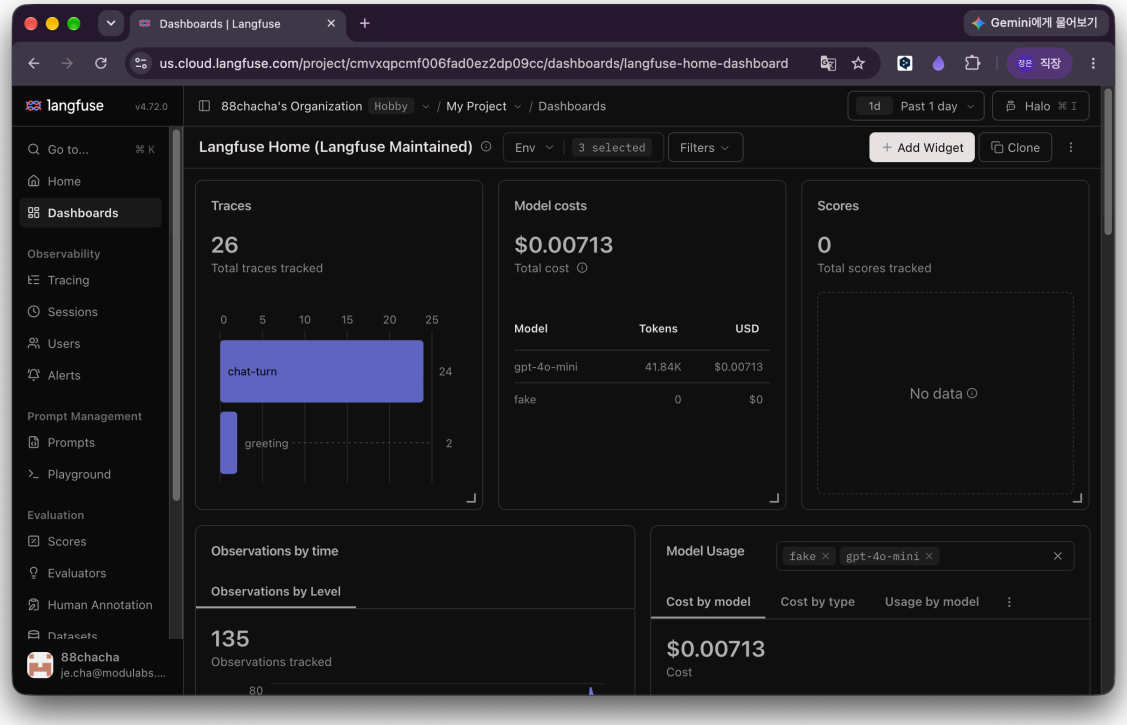


그림: Langfuse > Dashboards: 트레이스 수, 총비용, 모델별 비용 그래프가 보이는 기본 대시보드

결과 해석: 메시지 하나가 쓰는 호출

메시지 하나는 위기 표현이 아니면 도구 선택, 답 만들기, 기억 정리로 모델을 세 번 부릅니다. 기억 정리는 사용자가 보지 못하는 호출이지만 메시지마다 빠짐없이 비용이 들고, 요약되지 않은 메시지가 40개를 넘으면 `memory-summary` 호출이 한 번 더 붙습니다. 대화가 처음이거나 6시간 넘게 쉬었다가 화면을 열면 메시지를 보내지 않아도 `greeting` 이 모델을 부릅니다. "메시지 수 곱하기 호출 한 번"으로 예상한 금액이 청구서와 맞지 않는 이유가 여기에 있습니다.

가장 비싼 트레이스와 가장 싼 트레이스를 비교하면 `reply` 의 입력 토큰 차이가 가장 큰 경우가 많습니다. 답 만들기 입력에는 캐릭터 지시문, 기억 메모, 대화 요약, 최근 10턴의 대화가 들어가므로, 대화가 길어질수록 같은 짧은 메시지에도 입력 토큰이 커집니다. 메시지 한 건의 평균 비용은 나중에 가격을 정할 때 출발점이 되는 숫자이지만, 지금은 내가 보낸 몇 개의 메시지로 계산했으므로 정확하지 않습니다. 동료들이 실제로 쓴 트레이스가 쌓인 뒤 다시 계산해야 의미가 있습니다.

▼ 비용 칸이 비어 있을 때

가짜 LLM으로 띄웠다면 모델을 부르지 않았으니 토큰과 비용이 0입니다. 실제 모델인데 비용만 비어 있다면 Langfuse가 그 모델 이름을 가격표에서 찾지 못한 것입니다. Langfuse는 널리 쓰이는 OpenAI, Anthropic, Google 모델의 가격을 미리 알고 있어서 모델 이름으로 비용을 계산하는데, 목록에 없는 모델이나 이름이 조금 다르게 기록된 모델은 계산하지 못합니다. 이때는 Project Settings > Models 에서 모델 이름과 토큰당 가격을 직접 추가하면, 그 뒤에 들어오는 기록부터 비용이 계산됩니다.

객관식 퀴즈

Q. 하루 메시지 1,000건에 모델 호출 한 번 가격을 곱해 예상한 금액보다 청구서가 3배 가까이 나왔습니다. Langfuse에서 가장 먼저 확인할 것은 무엇인가요?

• 1. 트레이스 수가 1,000개인지
• 2. 트레이스 하나를 열어 모델을 부르는 관찰 단위가 몇 개이고 각각의 토큰과 비용이 얼마인지, 그리고 greeting 처럼 메시지 없이 생긴 트레이스가 있는지
• 3. 응답 지연 시간이 긴 트레이스
• 4. 점수가 낮은 트레이스

- •

  1. 트레이스 수가 1,000개인지

- •

  2. 트레이스 하나를 열어 모델을 부르는 관찰 단위가 몇 개이고 각각의 토큰과 비용이 얼마인지, 그리고 greeting 처럼 메시지 없이 생긴 트레이스가 있는지

- 3. 응답 지연 시간이 긴 트레이스
- 4. 점수가 낮은 트레이스

Chapter07

속성이 있어야 보이는 두 상황

한 동료는 "다음 주에 이사 간다고 말했는데 기억을 못 해요"라고 했습니다. 원인을 찾으려면 그 동료의 트레이스만 걸러 대화 순서대로 놓고, 이사 이야기를 한 메시지와 기억을 못 한 메시지를 찾아야 합니다. 그런데 지금 트레이스에는 누가 보냈는지가 없어서, 다섯 동료의 대화가 한 목록에 섞여 있습니다. 다른 동료가 말한 "업데이트한 뒤로 느려졌다"는 말이 맞는지 보려면 배포 버전별로 트레이스를 나눠 같은 숫자를 비교해야 하는데, 지금은 어느 리비전이 만든 트레이스인지 모르고 내 컴퓨터에서 시험한 트레이스까지 섞여 있습니다.

두 상황 모두 섹션 4에서 정한 속성을 트레이스에 붙여야 풀 수 있습니다. Langfuse SDK에는 한 번 정한 속성을 그 안의 모든 관찰 단위에 함께 붙이는 방법이 있으므로, 한 턴의 시작 지점에서 한 번만 정하면 됩니다.

요청 틀(빈칸은 직접 채운다):

```
같은 브랜치에서 Langfuse 트레이스에 속성을 붙여 줘. 공식 문서 근거를 알려 줘.
- user_id: 서버가 로그인으로 확인한 사용자 ID (이메일은 넣지 마)
- session_id: [사용자 ID + 한국 날짜 / 다른 기준:        ]
- environment: 환경변수 ENV 값 (local, staging, production)
- version: Cloud Run 이 넣어 주는 K_REVISION. 없으면 local
- metadata: 커밋(COMMIT_SHA), 고른 도구 이름, 도구 선택 확신도, 안전 응답 여부
- tags: [고른 도구 이름 / 안전 응답 / 필요한 것:        ]
- chat-turn 과 greeting 모두에 붙여 줘. 바꾸는 줄과 이유를 먼저 설명해 줘.
```

이사 이야기 다시 만들고 따라가기

속성을 붙였으면 내 컴퓨터에서 이사 이야기를 잊어버린 상황을 만듭니다. 이사 이야기를 하고, 몇 마디 뒤에 그 이야기를 기억하는지 묻습니다. 그다음 Langfuse의 사용자 화면과 세션 화면에서 그 대화를 찾아, 기억이 저장되는 단계와 쓰이는 단계를 차례로 봅니다.

직접 확인할 곳: 한 사람의 대화를 따라가기

1. `http://localhost:8080` 에서 "나 다음 주 토요일에 이사 가"를 보내고, 잡담을 두세 개 보낸 뒤 "내가 다음 주에 뭐 한다고 했지?"를 보냅니다.
2. Langfuse 트레이스 목록에서 User, Session, Environment, Version 열이 채워졌는지 봅니다. 로컬이므로 Environment는 `local`, Version은 `local` 입니다.
3. Users 화면에서 내 사용자 ID를 열어 트레이스 수와 비용이 한 사람 기준으로 모였는지 봅니다.
4. Sessions 화면에서 오늘 날짜의 세션을 열어 대화가 순서대로 이어지는지 봅니다.
5. 이사 이야기를 보낸 턴의 `memory-update` 출력에 이사 기억이 추가되었는지 봅니다.
6. 마지막 질문 턴의 `reply` 입력에서 `[기억 메모]` 부분에 그 기억이 들어 있는지 봅니다.

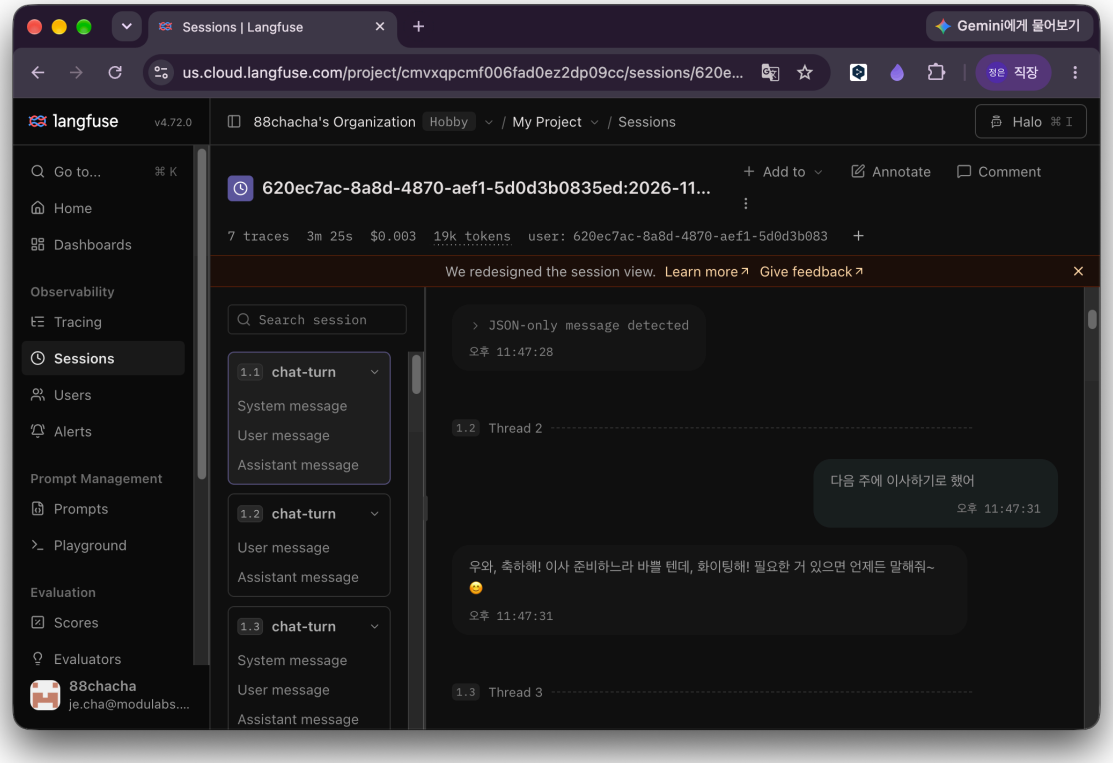


그림: Langfuse > Sessions > 오늘 날짜의 세션: 이사 이야기부터 마지막 질문까지 한 사람의 대화 트레이스가 순서대로 이어진 화면

결과 해석: 기억이 끊긴 자리

기억을 못 했다면 끊긴 자리는 세 곳 가운데 하나입니다. 첫째, `memory-update` 출력에 기억이 추가되지 않은 경우입니다. 모델이 기억할 거리로 보지 않았거나, 기억 정리 응답의 형식이 맞지 않아 그 턴의 기억 정리를 건너뛴 경우이고, 계좌 번호나 비밀번호처럼 민감한 패턴이

들어간 기억은 코드가 일부러 저장하지 않습니다. 둘째, 기억은 저장됐는데 `reply` 입력에 들어가지 않은 경우입니다. 답 만들기에는 최근 기억 30개까지만 넣으므로 기억이 아주 많은 사용자에게서 생길 수 있습니다. 셋째, 입력에 들어갔는데 답이 무시한 경우이고, 이때는 답 만들기 프롬프트를 봅니다.

한 가지 함정이 있습니다. 같은 날 몇 마디 사이의 대화라면 최근 10턴의 원문이 `reply` 입력에 함께 들어가므로, 기억 메모가 없어도 모델이 이사 이야기를 기억하는 것처럼 답할 수 있습니다. 그래서 오늘 실습에서 답이 맞았다고 해서 기억 정리가 잘 된 것은 아니고, 동료처럼 며칠 뒤에 물었을 때도 기억하는지는 `memory-update` 출력으로 확인해야 합니다. 실제 동료의 경우라면 Users 화면에서 그 동료를 열어 이사 이야기를 한 날의 세션을 찾고, 같은 순서로 보면 됩니다.

staging에 키 넣기

업데이트 뒤 느려진 답은 배포한 서비스의 버전을 비교해야 하므로 staging에서 확인합니다. 그러려면 staging 서비스가 Langfuse 키를 읽을 수 있어야 합니다. 비밀키는 콘솔에서 Secret Manager에 `langfuse-secret-key` 라는 이름으로 직접 만듭니다(보안 > Secret Manager > 보안 비밀 만들기). 키 값이 명령 기록이나 에이전트 대화에 남지 않게 하려는 것입니다. 그다음 런타임 서비스 계정이 이 시크릿 하나만 읽을 수 있게 권한을 주고, staging 서비스에 공개키와 접속 주소는 일반 환경변수로, 비밀키는 시크릿으로 연결합니다. 공개키 자리는 직접 붙여 넣습니다.

직접 실행할 명령: staging 서비스에 Langfuse 연결

```
gcloud secrets add-iam-policy-binding langfuse-secret-key --member "serviceAccount:fri
end-agent-runtime@<PROJECT_ID>.iam.gserviceaccount.com" --role roles/secretmanager.sec
retAccessor

gcloud run services update friend-agent-staging --region us-central1 \
  --update-env-vars LANGFUSE_PUBLIC_KEY=<공개키>,LANGFUSE_BASE_URL=https://us.cloud.lan
gfuse.com \
  --update-secrets LANGFUSE_SECRET_KEY=langfuse-secret-key:latest
```

코딩 에이전트에게 맡길 때: staging 서비스에 Langfuse 연결

```
friend-agent-runtime 서비스 계정에 Secret Manager 시크릿 langfuse-secret-key 하나에 대해서만
읽기 권한을 줘.
그다음 friend-agent-staging 서비스에 환경변수 LANGFUSE_PUBLIC_KEY, LANGFUSE_BASE_URL 을 추가
하고,
LANGFUSE_SECRET_KEY 는 langfuse-secret-key 최신 버전으로 연결해 줘. 공개키는 내가 붙여 넣을 테니
자리만 비워 둔 명령을 보여 줘.
다른 환경변수와 설정은 바꾸지 말고, 시크릿 값은 읽거나 출력하지 마.
```

실행 전에 환경변수 옵션이 `--update-env-vars` 인지(기존 값을 지우는 `--set-env-vars` 가 아닌지), 서비스 이름이 `friend-agent-staging` 인지 봅니다. Windows PowerShell에서는 줄 끝의 `\` 대신 백틱(`` ` ``)으로 줄을 잇거나 한 줄로 이어 씁니다. 서비스에 넣은 환경변수와 시크릿 연결은 다음 자동 배포에도 그대로 남습니다.

업데이트 앞뒤의 리비전 비교하기

"업데이트한 뒤로"를 확인하려면 리비전이 둘 있어야 합니다. 지금까지의 작업을 PR로 올려 staging에 머지하면 첫 리비전이 생기고, 같은 날 한 번 더 배포하면 두 번째 리비전이 생깁니다. 두 리비전에 비슷한 메시지를 보내고 Version으로 나눠 비교합니다.

직접 확인할 곳: staging에서 버전별로 보기

1. PR을 staging에 머지해 배포가 끝나면, staging 주소에서 도구를 쓰는 메시지와 잡담을 섞어 대여섯 개 보냅니다.
2. Langfuse 트레이스 목록 위의 환경 선택에서 `staging` 만 고르고, Version 열에 staging 리비전 이름이 찍혔는지 봅니다. 로컬에서 시험한 `local` 트레이스는 목록에서 빠져야 합니다.
3. 트레이스 하나를 열어 metadata에 커밋과 고른 도구, 확신도가 있는지 봅니다.
4. 새 리비전을 한 번 더 배포한 뒤 같은 메시지들을 보내고, Version으로 걸러 두 리비전의 트레이스가 나뉘어 보이는지 봅니다.
5. 리비전마다 트레이스의 Latency와 `reply` 의 첫 글자까지 걸린 시간을 비교합니다.

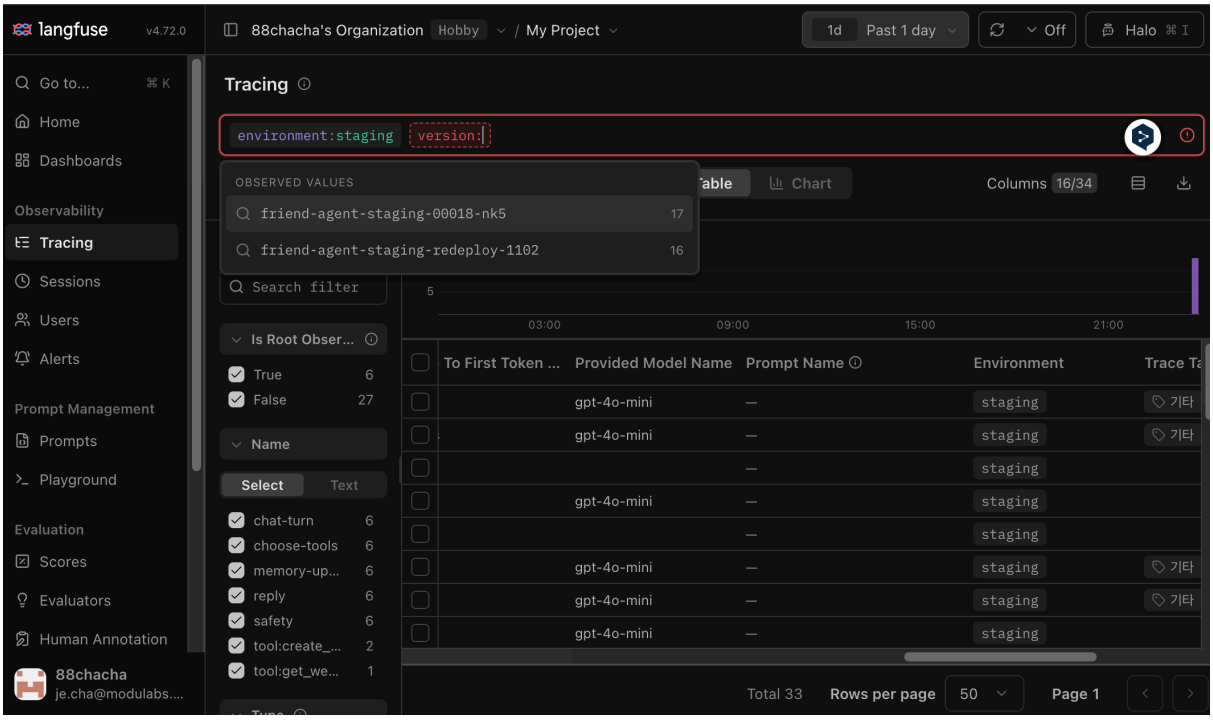


그림: Langfuse > Tracing > Traces, environment staging: 두 리비전이 Version 속성에 다르게 기록된 모습

결과 해석: 버전으로 나눠 보기

staging의 트레이스가 로컬 시험 트레이스와 섞이지 않고, 리비전마다 나뉘어 보입니다. 두 리비전의 숫자가 비슷하다면 "업데이트한 뒤로 느려졌다"는 말은 배포 때문이 아니라, 섹션 6에서 본 것처럼 도구를 쓰는 메시지였거나 LLM 제공사의 응답이 늦은 시간대였을 가능성이 큽니다. 새 리비전이 분명히 느리다면 섹션 6의 방법으로 어느 단계의 막대가 늘었는지 찾고, metadata의 커밋으로 그 리비전에 들어간 변경을 찾습니다. 새 리비전을 일부 요청에만 보내는 카나리 배포를 할 때도 이 Version 값이 판단 근거가 됩니다. production에도 같은 방법으로 키를 넣으면, 실제 사용자의 트레이스는 environment가 `production` 으로 따로 쌓입니다.

트레이스가 몇 분 늦게 나타나거나 일부가 빠진다면 Cloud Run의 특성 때문일 수 있습니다. Cloud Run은 요청을 처리하는 동안에만 CPU를 넉넉히 주는데, 섹션 2에서 본 것처럼 SDK는 트레이스를 모았다가 따로 보내기 때문입니다. 이때는 응답을 마칠 때 SDK의 `flush()` 를 부르도록 코딩 에이전트에게 맡깁니다. 대가는 응답이 아주 조금 늦어지는 것입니다.

Chapter08

언제였는지 모르는 별로인 답

동료는 "별로인 답이 몇 번 있었는데 언제였는지는 모르겠어요"라고 했습니다. 이제 트레이스는 사용자별로 걸러 볼 수 있지만, 그 동료의 트레이스가 수십 개라면 어느 것이 별로였는지 하나씩 읽어야 합니다. 사용자가 그 답을 본 자리에서 반응을 남기게 하고, 그 반응을 트레이스에 점수로 붙여 두면 "별로예요"가 붙은 트레이스부터 읽을 수 있습니다. 화면에는 이미 답 옆에 신고 버튼이 있지만, 지금은 서버 로그에 경고 한 줄을 남길 뿐입니다(`app/main.py` 의 `/api/report`). 여기에 좋아요와 별로예요 버튼을 더하고, 세 가지 반응을 모두 Langfuse 점수로 보냅니다.

어려운 부분은 화면의 메시지와 Langfuse 트레이스를 잇는 일입니다. 화면은 답의 메시지 ID(`message_id`)만 알고, Langfuse 점수는 트레이스 ID를 알아야 붙일 수 있습니다. 둘을 잇는 방법은 몇 가지가 있습니다. 답을 저장할 때 메시지의 메타데이터(`meta`)에 트레이스 ID를 함께 넣어 두고 반응이 오면 꺼내 쓰는 방법, 답 스트리밍이 끝날 때 보내는 `done` 이벤트에 트레이스 ID를 실어 화면이 들고 있게 하는 방법이 있습니다. 첫 번째는 다시 열었을 때도 이어지고, 두 번째는 저장 구조를 바꾸지 않습니다. 어느 쪽을 쓸지 정해 요청 틀에 적습니다.

요청 틀(빈칸은 직접 채운다):

```
같은 브랜치에서 사용자 반응을 Langfuse 점수로 보내게 해 줘. 공식 문서(점수 만들기) 근거를 알려 줘.
- 화면(static/index.html): 답 말풍선 옆에 좋아요, 별로예요 버튼을 더해 줘. 신고 버튼은 그대로 둬.
- 백엔드: 반응을 받는 API 를 [새로 만들기 / /api/report 를 넓히기] 로 해 줘.
- 메시지와 트레이스 잇기: [메시지 meta 에 트레이스 ID 저장 / done 이벤트에 트레이스 ID 싣기]
- 점수 이름과 값: user-feedback 은 좋아요 1, 별로예요 0 (BOOLEAN), report 는 신고 1 (BOOLEAN).
신고 사유는 comment 로.
- 남의 메시지에 점수를 매길 수 없게, 로그인한 사용자의 메시지인지 서버에서 확인해 줘.
- Langfuse 키가 없을 때는 점수 보내기를 건너뛰고 오류 없이 응답해 줘. 기존 테스트가 통과해야 해.
```

직접 확인할 곳: 점수가 붙는지 확인

1. 로컬에서 답 몇 개에 좋아요, 별로예요, 신고를 누릅니다. 섹션 5의 도쿄 답처럼 실제로 별로였던 답에 별로예요를 누릅니다.
2. Langfuse 트레이스 목록에서 방금 반응을 남긴 트레이스를 열어 `user-feedback`, `report` 점수가 붙었는지 봅니다. 목록에는 점수 열이 저절로 나타나지 않습니다.
3. 목록 위 검색창에 `scores."user-feedback":false` 를 넣어 별로예요가 붙은 트레이스만 걸러 봅니다. BOOLEAN 점수는 화면에서 0과 1이 아니라 false와 true로 나타납니다. User 필터를 함께 걸면 한 사람이 별로라고 한 답만 남습니다.
4. 다른 브라우저(다른 게스트)에서 남의 메시지 ID로 반응을 보내면 거절되는지, 코딩 에이전트에게 시험 방법을 물어 확인합니다.

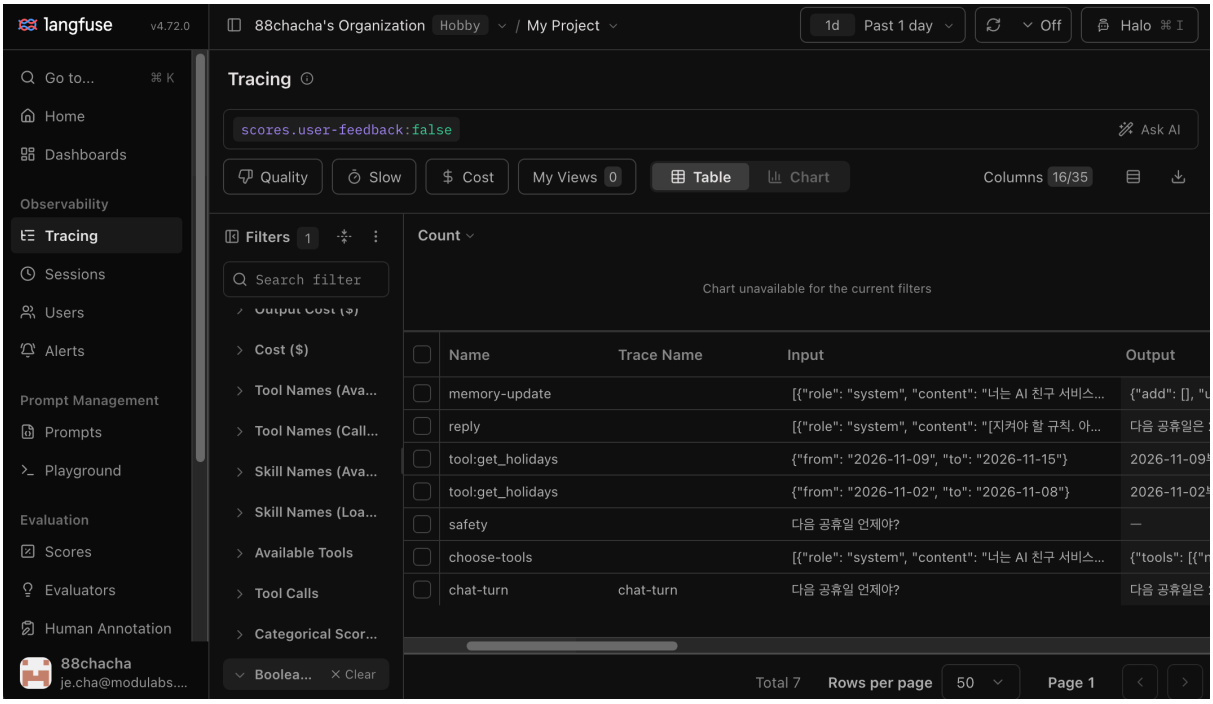


그림: Langfuse > Tracing > Traces, 검색창 필터: 검색창에 scores."user-feedback":false 를 넣어 별로예요가 붙은 트레이스만 걸러진 목록

결과 해석: 읽을 순서가 생긴 트레이스

이제 "별로예요"가 붙은 트레이스만 걸러, 섹션 5의 순서대로 처음 어긋난 단계를 찾을 수 있습니다. 그 동료가 앞으로 별로인 답을 만나면 그 자리에서 버튼을 누르기만 하면 되고, 언제였는지 기억할 필요가 없습니다. 다만 사용자 대부분은 버튼을 누르지 않습니다. 반응이 붙은 트레이스는 전체의 일부일 뿐이고, 화가 났을 때 더 자주 누르는 쪽으로 치우쳐 있습니다. 그래서 피드백 점수는 읽을 순서를 정하는 데 쓰고, 품질을 판단하는 숫자로는 사람이 직접 읽고 나눈 결과와 자동 채점을 함께 봅니다. 신고 점수는 앱 스토어에 올릴 때 요구하는 "AI가 만든 부적절한 내용을 신고하는 기능"의 기록이 되기도 합니다.

마지막으로 작업 브랜치를 PR로 올려 staging에 머지하고, staging에서 버튼이 동작하는지 한 번 더 확인합니다. production에는 키를 넣고 main에 머지할 때 함께 나갑니다.

▼ **점수가 다른 트레이스에 붙을 때**

메시지와 트레이스를 잇는 값이 어긋난 경우가 많습니다. 답을 저장하는 시점에 이미 끝난 트레이스의 ID를 쓰고 있지 않은지, 화면을 다시 열었을 때 불러온 지난 메시지에도 트레이스 ID가 함께 오는지 봅니다. 코딩 에이전트에게 "메시지 ID 하나를 골라, 그 메시지의 트레이스 ID가 어디서 정해지고 어디에 저장되는지 따라가며 보여 줘"라고 물어보면 끊긴 곳을 찾기 쉽습니다.

객관식 퀴즈

Q. "별로예요" 점수가 붙은 트레이스가 전체의 3%였습니다. 이 숫자에 대한 해석으로 가장 알맞은 것은 무엇인가요?

• 1. 나머지 97%의 답은 사용자가 만족한 것이다
• 2. 버튼을 누르는 사용자는 일부이고 불만이 있을 때 더 누르므로, 3%는 읽을 순서를 정하는 단서이고 전체 품질은 직접 읽고 나눈 결과와 함께 봐야 한다
• 3. 3%는 낮으므로 더 볼 필요가 없다
• 4. 점수가 잘못 붙은 것이므로 계측을 다시 해야 한다

- •

  1. 나머지 97%의 답은 사용자가 만족한 것이다

- •

  2. 버튼을 누르는 사용자는 일부이고 불만이 있을 때 더 누르므로, 3%는 읽을 순서를 정하는 단서이고 전체 품질은 직접 읽고 나눈 결과와 함께 봐야 한다

- •

  3. 3%는 낮으므로 더 볼 필요가 없다

-

  4. 점수가 잘못 붙은 것이므로 계측을 다시 해야 한다

Chapter09

오늘 배운 것

동료들의 말에서 기능으로. 섹션 1에서 동료들이 한 말은 모두 응답 코드 200 뒤에 숨은 일이었고, 서버 로그로는 어느 것에도 답할 수 없었습니다. 말마다 Langfuse의 어느 화면을 열지와 그 화면이 채워지려면 무엇을 미리 기록해야 하는지를 짝지었고, 그 짝이 계측, 속성, 점수를 더하는 순서가 되었습니다.

들어온 말	연 화면	찾은 것
엉뚱한 날씨 답	트레이스 하나의 단계별 입력과 출력	도구 결과와 답 만들기 사이에서 처음 어긋난 단계
가끔 느린 답	Latency 정렬과 시간 막대, 첫 글자까지의 시간	트레이스 지연 시간과 사용자가 기다린 시간의 차이, 느린 단계
메시지 수보다 많은 요금	generation별 토큰과 비용, 대시보드	메시지당 모델 호출 수와 메시지 없이 생긴 호출
잊어버린 이사 이야기	Users, Sessions, 기억 정리 출력	기억이 저장, 전달, 사용 가운데 어디서 끊겼는지
업데이트 뒤 느려진 답	environment와 version으로 거른 목록	리비전 사이에 숫자가 실제로 달라졌는지
언제였는지 모르는 별로인 답	점수로 거른 목록	먼저 읽을 트레이스

Langfuse의 동작 방식. SDK는 관찰 단위를 버퍼에 모았다가 백그라운드에서 한꺼번에 보내므로 사용자의 답은 늦어지지 않지만, 화면에는 몇 초 늦게 뜨고 짧은 스크립트는 끝에서 `flush()` 가 필요합니다. 서버는 받은 기록을 대기열과 워커를 거쳐 저장하고, Cloud와 셀프 호스팅은 이 서버를 누가 운영하느냐의 차이입니다.

구조를 먼저 정하는 계측. 어떤 단계를 어떤 관찰 단위로 남길지는 풀어야 할 문제에서 거꾸로 정했고, 세션을 무엇으로 볼지, 원문을 남길지와 무엇을 가릴지도 서비스를 아는 사람이 먼저 정했습니다. 코딩 에이전트에게는 그 구조와 공식 문서 근거, 키가 없을 때의 동작을 함께 요청했습니다.

주관식 퀴즈

Q. [회고] 섹션 1의 시작 질문에서 고른 동료의 말과 적어 둔 답을 다시 읽어 보세요. 오늘 그 상황을 Langfuse의 어느 화면에서 어떻게 확인했는지, 그리고 오늘 만든 관측으로도 아직 확인할 수 없는 것은 무엇인지 나눠 적어 보세요.

0/300**다음**

📚 오늘 나온 자료 모아보기

- Langfuse: Python SDK overview: SDK 설치, 환경변수, 클라이언트를 다시 볼 때 출발점입니다.
- Langfuse: Sessions: 세션 ID로 대화를 묶는 방법입니다.
- Langfuse: Environments: staging과 production 트레이스를 나누는 방법입니다.
- Langfuse: Releases and versioning: 버전별로 트레이스를 나눠 보는 방법입니다.
- Langfuse: Token and cost tracking: 모델 이름으로 비용을 계산하는 방식과 모델 정의를 직접 추가하는 방법입니다.
- Langfuse: Masking: 보내기 전에 개인정보를 가리는 방법입니다.
- Langfuse: Scores via SDK: 트레이스에 점수를 붙이는 방법입니다.

마지막으로

오늘 만든 관측은 무슨 일이 일어났는지를 보여 주지만, 그 일이 좋았는지 나빴는지는 아직 사람이 읽어야 압니다. 피드백 점수는 버튼을 누른 일부 사용자의 반응이라 한쪽으로 치우쳐 있고, 트레이스를 하나씩 읽는 방법은 수백 개가 쌓이면 감당할 수 없습니다. 오늘 문제를 다시 만들 때 보낸 메시지도 몇 개뿐이라, 느린 단계나 메시지 한 건의 평균 비용은 실제 사용자의 대화가 쌓인 뒤 다시 봐야 합니다. 그리고 트레이스에는 대화 원문이 올라가므로, 가린 패턴과 보관 기간, 사용자 안내를 서비스를 공개하기 전에 한 번 더 점검해야 합니다. 아침에는 동료들의 말에 서버 로그만 뒤적였다면, 지금은 목록의 문제마다 어느 화면을 열어 무엇을 볼지 말할 수 있고, 새로 들어오는 문제에도 같은 방법으로 확인할 곳을 고를 수 있습니다.